

2. Apply Algorithm 4.12 to the kernel of each set of LR(0) items and grammar symbol X to determine which lookaheads are spontaneously generated for kernel items in $\text{goto}(I, X)$, and from which items in I lookaheads are propagated to kernel items in $\text{goto}(I, X)$.
3. Initialize a table that gives, for each kernel item in each set of items, the associated lookaheads. Initially, each item has associated with it only those lookaheads that we determined in (2) were generated spontaneously.
4. Make repeated passes over the kernel items in all sets. When we visit an item i , we look up the kernel items to which i propagates its lookaheads, using information tabulated in (2). The current set of lookaheads for i is added to those already associated with each of the items to which i propagates its lookaheads. We continue making passes over the kernel items until no more new lookaheads are propagated. $\square$

Example 4.47. Let us construct the kernels of the LALR(1) items for the grammar in the previous example. The kernels of the LR(0) items were shown in Fig. 4.42. When we apply Algorithm 4.12 to the kernel of set of items I_0 , we compute $\text{closure}(\{[S' \rightarrow \cdot S, \#]\})$, which is

$$\begin{aligned} S' &\rightarrow \cdot S, \# \\ S &\rightarrow \cdot L = R, \# \\ S &\rightarrow \cdot R, \# \\ L &\rightarrow \cdot *R, \# / = \\ L &\rightarrow \cdot \text{id}, \# / = \\ R &\rightarrow \cdot L, \# \end{aligned}$$

Two items in this closure cause lookaheads to be generated spontaneously. Item $[L \rightarrow \cdot *R, =]$ causes lookahead $=$ to be spontaneously generated for kernel item $L \rightarrow * \cdot R$ in I_4 and item $[L \rightarrow \cdot \text{id}, =]$ causes $=$ to be spontaneously generated for kernel item $L \rightarrow \text{id} \cdot$ in I_5 .

The pattern of propagation of lookaheads among the kernel items determined in step (2) of Algorithm 4.13 is summarized in Fig. 4.44. For example, the gotos of I_0 on symbols $S, L, R, *,$ and id are respectively $I_1, I_2, I_3, I_4,$ and I_5 . For I_0 we computed only the closure of the lone kernel item $[S' \rightarrow \cdot S, \#]$. Thus, $S' \rightarrow \cdot S$ propagates its lookahead to each kernel item in I_1 through I_5 .

In Fig. 4.45, we show steps (3) and (4) of Algorithm 4.13. The column labeled INIT shows the spontaneously generated lookaheads for each kernel item. On the first pass, the lookahead $\$$ propagates from $S' \rightarrow S$ in I_0 to the six items listed in Fig. 4.44. The lookahead $=$ propagates from $L \rightarrow * \cdot R$ in I_4 to items $L \rightarrow *R \cdot$ in I_7 and $R \rightarrow L \cdot$ in I_8 . It also propagates to itself and to $L \rightarrow \text{id} \cdot$ in I_5 , but these lookaheads are already present. In the second and third passes, the only new lookahead propagated is $\$,$ discovered for the successors of I_2 and I_4 on pass 2 and for the successor of I_6 on pass 3. No new lookaheads are propagated on pass 4, so the final set of lookaheads is shown

FROM	TO
$I_0: S' \rightarrow \cdot S$	$I_1: S' \rightarrow S \cdot$ $I_2: S \rightarrow L \cdot = R$ $I_2: R \rightarrow L \cdot$ $I_3: S \rightarrow R \cdot$ $I_4: L \rightarrow * \cdot R$ $I_5: L \rightarrow \text{id} \cdot$
$I_2: S \rightarrow L \cdot = R$	$I_6: S \rightarrow L = \cdot R$
$I_4: L \rightarrow * \cdot R$	$I_4: L \rightarrow * \cdot R$ $I_5: L \rightarrow \text{id} \cdot$ $I_7: L \rightarrow * R \cdot$ $I_8: R \rightarrow L \cdot$
$I_6: S \rightarrow L = \cdot R$	$I_4: L \rightarrow * \cdot R$ $I_5: L \rightarrow * \cdot R$ $I_8: R \rightarrow L \cdot$ $I_9: S \rightarrow L = R \cdot$

Fig. 4.44. Propagation of lookaheads.

in the rightmost column of Fig. 4.45.

Note that the shift/reduce conflict found in Example 4.39 using the SLR method has disappeared with the LALR technique. The reason is that only lookahead \$ is associated with $R \rightarrow L \cdot$ in I_2 , so there is no conflict with the parsing action of shift on = generated by item $S \rightarrow L \cdot = R$ in I_2 . $\square$

Compaction of LR Parsing Tables

A typical programming language grammar with 50 to 100 terminals and 100 productions may have an LALR parsing table with several hundred states. The action function may easily have 20,000 entries, each requiring at least 8 bits to encode. Clearly a more efficient encoding than a two-dimensional array may be important. We shall briefly mention a few techniques that have been used to compress the action and goto fields of an LR parsing table.

One useful technique for compacting the action field is to recognize that usually many rows of the action table are identical. For example, in Fig. 4.40, states 0 and 3 have identical action entries, and so do 2 and 6. We can therefore save considerable space, at little cost in time, if we create a pointer for each state into a one-dimensional array. Pointers for states with the same actions point to the same location. To access information from this array, we assign each terminal a number from zero to one less than the number of terminals, and we use this integer as an offset from the pointer value for each state. In a given state, the parsing action for the i th terminal will be found i locations past the pointer value for that state.

SET	ITEM	LOOKAHEADS			
		INIT	PASS 1	PASS 2	PASS 3
I_0	$S' \rightarrow \cdot S$	\$	\$	\$	\$
I_1	$S' \rightarrow S \cdot$		\$	\$	\$
I_2	$S \rightarrow L \cdot = R$		\$	\$	\$
I_2	$R \rightarrow L \cdot$		\$	\$	\$
I_3	$S \rightarrow R \cdot$		\$	\$	\$
I_4	$L \rightarrow * \cdot R$	=	=/\$	=/\$	=/\$
I_5	$L \rightarrow id \cdot$	=	=/\$	=/\$	=/\$
I_6	$S \rightarrow L = \cdot R$			\$	\$
I_7	$L \rightarrow * R \cdot$		=	=/\$	=/\$
I_8	$R \rightarrow L \cdot$		=	=/\$	=/\$
I_9	$S \rightarrow L = R \cdot$				\$

Fig. 4.45. Computation of lookaheads.

Further space efficiency can be achieved at the expense of a somewhat slower parser (generally considered a reasonable trade, since an LR-like parser consumes only a small fraction of the total compilation time) by creating a list for the actions of each state. The list consists of (terminal-symbol, action) pairs. The most frequent action for a state can be placed at the end of the list, and in place of a terminal we may use the notation "any," meaning that if the current input symbol has not been found so far on the list, we should do that action no matter what the input is. Moreover, error entries can safely be replaced by reduce actions, for further uniformity along a row. The errors will be detected later, before a shift move.

Example 4.48. Consider the parsing table of Fig. 4.31. First, note that the actions for states 0, 4, 6, and 7 agree. We can represent them all by the list:

SYMBOL	ACTION
id	s5
(	s4
any	error

State 1 has a similar list:

+	s6
\$	acc
any	error

In state 2, we can replace the error entries by *r2*, so reduction by production 2 will occur on any input but ***. Thus the list for state 2 is:

<i>*</i>	<i>s7</i>
any	<i>r2</i>

State 3 has only error and *r4* entries. We can replace the former by the latter, so the list for state 3 consists of only the pair (any, *r4*). States 5, 10, and 11 can be treated similarly. The list for state 8 is:

+	<i>s6</i>
)	<i>s11</i>
any	error

and for state 9:

<i>*</i>	<i>s7</i>
any	<i>r1</i>

□

We can also encode the *goto* table by a list, but here it appears more efficient to make a list of pairs for each nonterminal *A*. Each pair on the list for *A* is of the form (*current_state*, *next_state*), indicating

$$\text{goto}[\text{current_state}, A] = \text{next_state}$$

This technique is useful because there tend to be rather few states in any one column of the *goto* table. The reason is that the *goto* on nonterminal *A* can only be a state derivable from a set of items in which some items have *A* immediately to the left of a dot. No set has items with *X* and *Y* immediately to the left of a dot if *X* ≠ *Y*. Thus, each state appears in at most one *goto* column.

For more space reduction, we note that the error entries in the *goto* table are never consulted. We can therefore replace each error entry by the most common non-error entry in its column. This entry becomes the default; it is represented in the list for each column by one pair with "any" in place of current-state.

Example 4.49. Consider Fig. 4.31 again. The column for *F* has entry 10 for state 7, and all other entries are either 3 or error. We may replace error by 3 and create for column *F* the list:

<i>current_state</i>	<i>next_state</i>
7	10
any	3

Similarly, a suitable list for column *T* is:

6	9
any	2

For column *E* we may choose either 1 or 8 to be the default; two entries are necessary in either case. For example, we might create for column *E* the list:

4	8	
any	1	□

If the reader totals up the number of entries in the lists created in this example and the previous one, and then adds the pointers from states to action lists and from nonterminals to next-state lists, he will not be impressed with the space savings over the matrix implementation of Fig. 4.31. We should not be misled by this small example, however. For practical grammars, the space needed for the list representation is typically less than ten percent of that needed for the matrix representation.

We should also point out that the table-compression methods for finite automata that were discussed in Section 3.9 can also be used to represent LR parsing tables. Application of these methods is discussed in the exercises.

4.8 USING AMBIGUOUS GRAMMARS

It is a theorem that every ambiguous grammar fails to be LR, and thus is not in any of the classes of grammars discussed in the previous section. Certain types of ambiguous grammars, however, are useful in the specification and implementation of languages, as we shall see in this section. For language constructs like expressions an ambiguous grammar provides a shorter, more natural specification than any equivalent unambiguous grammar. Another use of ambiguous grammars is in isolating commonly occurring syntactic constructs for special case optimization. With an ambiguous grammar, we can specify the special case constructs by carefully adding new productions to the grammar.

We should emphasize that although the grammars we use are ambiguous, in all cases we specify disambiguating rules that allow only one parse tree for each sentence. In this way, the overall language specification still remains unambiguous. We also stress that ambiguous constructs should be used sparingly and in a strictly controlled fashion; otherwise, there can be no guarantee as to what language is recognized by a parser.

Using Precedence and Associativity to Resolve Parsing Action Conflicts

Consider expressions in programming languages. The following grammar for arithmetic expressions with operators $+$ and $*$

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id} \quad (4.22)$$

is ambiguous because it does not specify the associativity or precedence of the operators $+$ and $*$. The unambiguous grammar

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \quad (4.23)$$

generates the same language, but gives $+$ a lower precedence than $*$, and makes both operators left-associative. There are two reasons why we might

want to use grammar (4.22) instead of (4.23). First, as we shall see, we can easily change the associativities and precedence levels of the operators $+$ and $*$ without disturbing the productions of (4.22) or the number of states in the resulting parser. Second, the parser for (4.23) will spend a substantial fraction of its time reducing by the productions $E \rightarrow T$ and $T \rightarrow F$, whose sole function is to enforce associativity and precedence. The parser for (4.22) will not waste time reducing by these *single* productions, as they are called.

$I_0:$	$E' \rightarrow \cdot E$ $E \rightarrow \cdot E + E$ $E \rightarrow \cdot E * E$ $E \rightarrow \cdot (E)$ $E \rightarrow \cdot id$	$I_5:$	$E \rightarrow E * \cdot E$ $E \rightarrow \cdot E + E$ $E \rightarrow \cdot E * E$ $F \rightarrow \cdot (E)$ $E \rightarrow \cdot id$
$I_1:$	$E' \rightarrow E \cdot$ $E \rightarrow E \cdot + E$ $E \rightarrow E \cdot * E$	$I_6:$	$E \rightarrow (E \cdot)$ $E \rightarrow E \cdot + E$ $E \rightarrow E \cdot * E$
$I_2:$	$E \rightarrow (\cdot E)$ $E \rightarrow \cdot E + E$ $E \rightarrow \cdot E * E$ $E \rightarrow \cdot (E)$ $E \rightarrow \cdot id$	$I_7:$	$E \rightarrow E + \cdot E$ $E \rightarrow E \cdot + E$ $E \rightarrow E \cdot * E$
$I_3:$	$E \rightarrow id \cdot$	$I_8:$	$E \rightarrow E * \cdot E$ $E \rightarrow E \cdot + E$ $E \rightarrow E \cdot * E$
$I_4:$	$E \rightarrow E + \cdot E$ $E \rightarrow \cdot E + E$ $E \rightarrow \cdot E * E$ $E \rightarrow \cdot (E)$ $E \rightarrow \cdot id$	$I_9:$	$E \rightarrow (E \cdot)$

Fig. 4.46. Sets of LR(0) items for augmented grammar (4.22).

The sets of LR(0) items for (4.22) augmented by $E' \rightarrow E$ are shown in Fig. 4.46. Since grammar (4.22) is ambiguous, parsing action conflicts will be generated when we try to produce an LR parsing table from the sets of items. The states corresponding to sets of items I_7 and I_8 generate these conflicts. Suppose we use the SLR approach to constructing the parsing action table. The conflict generated by I_7 between reduction by $E \rightarrow E + E$ and shift on $+$ and $*$ cannot be resolved because $+$ and $*$ are each in FOLLOW(E). Thus both actions would be called for on inputs $+$ and $*$. A similar conflict is generated by I_8 , between reduction by $E \rightarrow E * E$ and shift on inputs $+$ and $*$. In fact, each of our LR parsing table construction methods will generate these conflicts.

However, these problems can be resolved using the precedence and associativity information for $+$ and $*$. Consider the input $\text{id} + \text{id} * \text{id}$, which causes a parser based on Fig. 4.46 to enter state 7 after processing $\text{id} + \text{id}$; in particular the parser reaches a configuration

STACK	INPUT
$0 E 1 + 4 E 7$	$* \text{id} \$$

Assuming that $*$ takes precedence over $+$, we know the parser should shift $*$ onto the stack, preparing to reduce the $*$ and its surrounding id 's to an expression. This is what the SLR parser of Fig. 4.31 for the same language would do, and it is what an operator-precedence parser would do. On the other hand, if $+$ takes precedence over $*$, we know the parser should reduce $E + E$ to E . Thus the relative precedence of $+$ followed by $*$ uniquely determines how the parsing action conflict between reducing $E \rightarrow E + E$ and shifting on $*$ in state 7 should be resolved.

If the input had been $\text{id} + \text{id} + \text{id}$, instead, the parser would still reach a configuration in which it had stack $0E1+4E7$ after processing input $\text{id} + \text{id}$. On input $+$ there is again a shift/reduce conflict in state 7. Now, however, the associativity of the $+$ operator determines how this conflict should be resolved. If $+$ is left-associative, the correct action is to reduce by $E \rightarrow E + E$. That is, the id 's surrounding the first $+$ must be grouped first. Again this choice coincides with what the SLR or operator-precedence parsers would do for the grammar of Example 4.34.

In summary, assuming $+$ is left-associative, the action of state 7 on input $+$ should be to reduce by $E \rightarrow E + E$, and assuming that $*$ takes precedence over $+$, the action of state 7 on input $*$ should be to shift. Similarly, assuming that $*$ is left-associative and takes precedence over $+$, we can argue that state 8, which can appear on top of the stack only when $E * E$ are the top three grammar symbols, should have action reduce $E \rightarrow E * E$ on both $+$ and $*$ inputs. In the case of input $+$, the reason is that $*$ takes precedence over $+$, while in the case of input $*$, the rationale is that $*$ is left-associative.

Proceeding in this way, we obtain the LR parsing table shown in Fig. 4.47. Productions 1–4 are $E \rightarrow E + E$, $E \rightarrow E * E$, $E \rightarrow (E)$, and $E \rightarrow \text{id}$, respectively. It is interesting that a similar parsing action table would be produced by eliminating the reductions by the single productions $E \rightarrow T$ and $T \rightarrow F$ from the SLR table for grammar (4.23) shown in Fig. 4.31. Ambiguous grammars like (4.22) can be handled in a similar way in the context of LALR and canonical LR parsing.

The “Dangling-else” Ambiguity

Consider again the following grammar for conditional statements:

$$\begin{aligned} \text{stmt} &\rightarrow \text{if expr then stmt else stmt} \\ &\quad | \text{if expr then stmt} \\ &\quad | \text{other} \end{aligned}$$

STATE	action						goto
	id	+	*	(	)	\$	E
0	s3			s2			1
1		s4	s5			acc	
2	s3			s2			6
3		r4	r4		r4	r4	
4	s3			s2			8
5	s3			s2			8
6		s4	s5		s9		
7		r1	s5		r1	r1	
8		r2	r2		r2	r2	
9		r3	r3		r3	r3	

Fig. 4.47. Parsing table for grammar (4.22).

As we noted in Section 4.3, this grammar is ambiguous because it does not resolve the dangling-else ambiguity. To simplify the discussion, let us consider an abstraction of the above grammar, where *i* stands for **if expr then**, *e* stands for **else**, and *a* stands for "all other productions." We can then write the grammar, with augmenting production $S' \rightarrow S$, as:

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow iSeS \mid iS \mid a \end{aligned} \quad (4.24)$$

The sets of LR(0) items for grammar (4.24) are shown in Fig. 4.48. The ambiguity in (4.24) gives rise to a shift/reduce conflict in I_4 . There, $S \rightarrow iS \cdot eS$ calls for a shift of *e* and, since $\text{FOLLOW}(S) = \{e, \$\}$, item $S \rightarrow iS \cdot$ calls for reduction by $S \rightarrow iS$ on input *e*.

Translating back to the **if** $\cdots$ **then** $\cdots$ **else** terminology, given

if expr then stmt

on the stack and **else** as the first input symbol, should we shift **else** onto the stack (i.e., shift *e*) or reduce **if expr then stmt** to *stmt* (i.e., reduce by $S \rightarrow iS$)? The answer is that we should shift **else**, because it is "associated" with the previous **then**. In the terminology of grammar (4.24), the *e* on the input, standing for **else**, can only form part of the right side beginning with the *iS* on the top of the stack. If what follows *e* on the input cannot be parsed as an *S*, completing right side *iSeS*, then it can be shown that there is no other parse possible.

We are drawn to the conclusion that the shift/reduce conflict in I_4 should be resolved in favor of shift on input *e*. The SLR parsing table constructed from the sets of items of Fig. 4.48, using this resolution of the parsing action conflict in I_4 on input *e*, is shown in Fig. 4.49. Productions 1 through 3 are $S \rightarrow iSeS$, $S \rightarrow iS$, and $S \rightarrow a$, respectively.

$I_0:$	$S' \rightarrow \cdot S$ $S \rightarrow \cdot iSeS$ $S \rightarrow \cdot iS$ $S \rightarrow \cdot a$	$I_3:$	$S \rightarrow a \cdot$
$I_1:$	$S' \rightarrow S \cdot$	$I_4:$	$S \rightarrow iS \cdot eS$ $S \rightarrow iS \cdot$
$I_2:$	$S \rightarrow i \cdot SeS$ $S \rightarrow i \cdot S$ $S \rightarrow \cdot iSeS$ $S \rightarrow \cdot iS$ $S \rightarrow \cdot a$	$I_5:$	$S \rightarrow iSe \cdot S$ $S \rightarrow \cdot iSeS$ $S \rightarrow \cdot iS$ $S \rightarrow \cdot a$
		$I_6:$	$S \rightarrow iSeS \cdot$

Fig. 4.48. LR(0) states for augmented grammar (4.24).

STATE	action				goto
	<i>i</i>	<i>e</i>	<i>a</i>	\$	<i>S</i>
0	s2		s3		1
1				acc	
2	s2		s3		4
3		r3		r3	
4		s5		r2	
5	s2		s3		6
6		r1		r1	

Fig. 4.49. LR parsing table for abstract “dangling-else” grammar.

For example, on input *iiaea*, the parser makes the moves shown in Fig. 4.50, corresponding to the correct resolution of the “dangling-else.” At line (5), state 4 selects the shift action on input *e*, whereas at line (9), state 4 calls for reduction by $S \rightarrow iS$ on \$ input.

By way of comparison, if we are unable to use an ambiguous grammar to specify conditional statements, then we would have to use a bulkier grammar along the lines of (4.9).

Ambiguities from Special-Case Productions

Our final example suggesting the usefulness of ambiguous grammars occurs if we introduce an additional production to specify a special case of a syntactic construct generated in a more general fashion by the rest of the grammar. When we add the extra production, we generate a parsing action conflict. We can often resolve the conflict satisfactorily by a disambiguating rule that says

	STACK	INPUT
(1)	0	<i>iaea</i> \$
(2)	0 <i>i</i> 2	<i>iaea</i> \$
(3)	0 <i>i</i> 2 <i>i</i> 2	<i>aea</i> \$
(4)	0 <i>i</i> 2 <i>i</i> 2 <i>a</i> 3	<i>ea</i> \$
(5)	0 <i>i</i> 2 <i>i</i> 2 <i>S</i> 4	<i>ea</i> \$
(6)	0 <i>i</i> 2 <i>i</i> 2 <i>S</i> 4 <i>e</i> 5	<i>a</i> \$
(7)	0 <i>i</i> 2 <i>i</i> 2 <i>S</i> 4 <i>e</i> 5 <i>a</i> 3	\$
(8)	0 <i>i</i> 2 <i>i</i> 2 <i>S</i> 4 <i>e</i> 5 <i>S</i> 6	\$
(9)	0 <i>i</i> 2 <i>S</i> 4	\$
(10)	0 <i>S</i> 1	\$

Fig. 4.50. Parsing actions taken on input *iaea*.

reduce by the special-case production. The semantic action associated with the additional production then allows the special case to be handled by a more specific mechanism.

An interesting use of special-case productions was made by Kernighan and Cherry [1975] in their equation-typesetting preprocessor EQN, which was used to help typeset this book. In EQN, the syntax of a mathematical expression is described by a grammar that uses a subscript operator **sub** and a superscript operator **sup**, as shown in the grammar fragment (4.25). Braces are used by the preprocessor to bracket compound expressions, and *c* is used as a token representing any string of text.

- (1) $E \rightarrow E \text{ sub } E \text{ sup } E$
 - (2) $E \rightarrow E \text{ sub } E$
 - (3) $E \rightarrow E \text{ sup } E$
 - (4) $E \rightarrow \{ E \}$
 - (5) $E \rightarrow c$
- (4.25)

Grammar (4.25) is ambiguous for several reasons. The grammar does not specify the associativity and precedence of the operators **sub** and **sup**. Even if we resolve the ambiguities arising from the associativity and precedence of the **sub** and **sup**, say by making these two operators of equal precedence and right associative, the grammar will still be ambiguous. This is because production (1) isolates a special case of expressions generated by productions (2) and (3), namely expressions of the form $E \text{ sub } E \text{ sup } E$. The reason for treating expressions of this form specially is that many typesetters would prefer to typeset an expression like $a \text{ sub } i \text{ sup } 2$ as a_i^2 rather than as a_i^2 . By merely adding a special case production, Kernighan and Cherry were able to get EQN to produce this special case output.

To see how this kind of ambiguity can be treated in the LR setting, let us construct an SLR parser for grammar (4.25). The sets of LR(0) items for this

$I_0:$	$E' \rightarrow \cdot E$ $E \rightarrow \cdot E \text{ sub } E \text{ sup } E$ $E \rightarrow \cdot E \text{ sub } E$ $E \rightarrow \cdot E \text{ sup } E$ $E \rightarrow \cdot \{ E \}$ $E \rightarrow \cdot c$	$I_6:$	$E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E$ $E \rightarrow E \cdot \text{sup } E$ $E \rightarrow \{ E \cdot \}$
$I_1:$	$E' \rightarrow E \cdot$ $E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E$ $E \rightarrow E \cdot \text{sup } E$	$I_7:$	$E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E$ $E \rightarrow E \cdot \text{sub } E \cdot$ $E \rightarrow E \cdot \text{sup } E$
$I_2:$	$E \rightarrow \{ \cdot E \}$ $E \rightarrow \cdot E \text{ sub } E \text{ sup } E$ $E \rightarrow \cdot E \text{ sub } E$ $E \rightarrow \cdot E \text{ sup } E$ $E \rightarrow \cdot \{ E \}$ $E \rightarrow \cdot c$	$I_8:$	$E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E$ $E \rightarrow E \cdot \text{sup } E$ $E \rightarrow E \cdot \text{sup } E \cdot$
$I_3:$	$E \rightarrow c \cdot$	$I_9:$	$E \rightarrow \{ E \} \cdot$
$I_4:$	$E \rightarrow E \text{ sub } \cdot E \text{ sup } E$ $E \rightarrow E \text{ sub } \cdot E$ $E \rightarrow \cdot E \text{ sub } E \text{ sup } E$ $E \rightarrow \cdot E \text{ sub } E$ $E \rightarrow \cdot E \text{ sup } E$ $E \rightarrow \cdot \{ E \}$ $E \rightarrow \cdot c$	$I_{10}:$	$E \rightarrow E \text{ sub } E \text{ sup } \cdot E$ $E \rightarrow E \text{ sup } \cdot E$ $E \rightarrow \cdot E \text{ sub } E \text{ sup } E$ $E \rightarrow \cdot E \text{ sub } E$ $E \rightarrow \cdot E \text{ sup } E$ $E \rightarrow \cdot \{ E \}$ $E \rightarrow \cdot c$
$I_5:$	$E \rightarrow E \text{ sup } \cdot E$ $E \rightarrow \cdot E \text{ sub } E \text{ sup } E$ $E \rightarrow \cdot E \text{ sub } E$ $E \rightarrow \cdot E \text{ sup } E$ $E \rightarrow \cdot \{ E \}$ $E \rightarrow \cdot c$	$I_{11}:$	$E \rightarrow E \cdot \text{sub } E \text{ sup } E$ $E \rightarrow E \cdot \text{sub } E \text{ sup } E \cdot$ $E \rightarrow E \cdot \text{sub } E$ $E \rightarrow E \cdot \text{sup } E$ $E \rightarrow E \cdot \text{sup } E \cdot$

Fig. 4.51. LR(0) sets of items for grammar (4.25).

grammar are shown in Fig. 4.51. In this collection, three sets of items yield parsing action conflicts. I_7 , I_8 , and I_{11} generate shift/reduce conflicts on the tokens **sub** and **sup** because the associativity and precedence of these operators have not been specified. We resolve these parsing action conflicts when we make **sub** and **sup** of equal precedence and right-associative. Thus, shift is preferred in each case.

I_{11} also generates a reduce/reduce conflict on inputs $\}$ and $\$$ between the two productions

$$\begin{aligned} E &\rightarrow E \text{ sub } E \text{ sup } E \\ E &\rightarrow E \text{ sup } E \end{aligned}$$

State I_{11} will be on top of the stack when we have seen an input that has been reduced to $E \text{ sub } E \text{ sup } E$ on the stack. If we resolve the reduce/reduce conflict in favor of production (1), we shall treat an equation of the form $E \text{ sub } E \text{ sup } E$ as a special case. Using these disambiguating rules, we obtain the SLR parsing table show in Fig. 4.52.

STATE	action						goto
	sub	sup	{	}	c	\$	E
0			s2		s3		1
1	s4	s5				acc	
2			s2		s3		6
3	r5	r5		r5		r5	
4			s2		s3		7
5			s2		s3		8
6	s4	s5		s9			
7	s4	s10		r2		r2	
8	s4	s5		r3		r3	
9	r4	r4		r4		r4	
10			s2		s3		11
11	s4	s5		r1		r1	

Fig. 4.52. Parsing table for grammar (4.25).

Writing unambiguous grammars that factor out special case syntactic constructs is very difficult. To appreciate how difficult this is, the reader is invited to construct an equivalent unambiguous grammar for (4.25) that isolates expressions of the form $E \text{ sub } E \text{ sup } E$.

Error Recovery in LR Parsing

An LR parser will detect an error when it consults the parsing action table and finds an error entry. Errors are never detected by consulting the goto table. Unlike an operator-precedence parser, an LR parser will announce error as soon as there is no valid continuation for the portion of the input thus far scanned. A canonical LR parsing will never make even a single reduction before announcing an error. The SLR and LALR parsers may make several reductions before announcing an error, but they will never shift an erroneous input symbol onto the stack.

In LR parsing, we can implement panic-mode error recovery as follows.

We scan down the stack until a state s with a goto on a particular nonterminal A is found. Zero or more input symbols are then discarded until a symbol a is found that can legitimately follow A . The parser then stacks the state $\text{goto}[s, A]$ and resumes normal parsing. There might be more than one choice for the nonterminal A . Normally these would be nonterminals representing major program pieces, such as an expression, statement, or block. For example, if A is the nonterminal *stmt*, a might be semicolon or *end*.

This method of recovery attempts to isolate the phrase containing the syntactic error. The parser determines that a string derivable from A contains an error. Part of that string has already been processed, and the result of this processing is a sequence of states on top of the stack. The remainder of the string is still on the input, and the parser attempts to skip over the remainder of this string by looking for a symbol on the input that can legitimately follow A . By removing states from the stack, skipping over the input, and pushing $\text{goto}[s, A]$ on the stack, the parser pretends that it has found an instance of A and resumes normal parsing.

Phrase-level recovery is implemented by examining each error entry in the LR parsing table and deciding on the basis of language usage the most likely programmer error that would give rise to that error. An appropriate recovery procedure can then be constructed; presumably the top of the stack and/or first input symbols would be modified in a way deemed appropriate for each error entry.

Compared with operator-precedence parsers, the design of specific error-handling routines for an LR parser is relatively easy. In particular, we do not have to worry about faulty reductions; any reduction called for by an LR parser is surely correct. Thus we may fill in each blank entry in the action field with a pointer to an error routine that will take an appropriate action selected by the compiler designer. The actions may include insertion or deletion of symbols from the stack or the input or both, or alteration and transposition of input symbols, just as for the operator-precedence parser. Like that parser, we must make our choices without allowing the possibility that the LR parser will get into an infinite loop. A strategy that assures at least one input symbol will be removed or eventually shifted, or that the stack will eventually shrink if the end of the input has been reached, is sufficient in this regard. Popping a stack state that covers a nonterminal should be avoided, because this modification eliminates from the stack a construct that has already been successfully parsed.

Example 4.50. Consider again the expression grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

Figure 4.53 shows the LR parsing table from Fig. 4.47 for this grammar, modified for error detection and recovery. We have changed each state that calls for a particular reduction on some input symbols by replacing error entries in that state by the reduction. This change has the effect of postponing

the error detection until one or more reductions are made, but the error will still be caught before any shift move takes place. The remaining blank entries from Fig. 4.47 have been replaced by calls to error routines.

STATE	action						goto
	id	+	*	(	)	\$	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

Fig. 4.53. LR parsing table with error routines.

The error routines are as follows. The similarity of these actions and the errors they represent to the error actions in Example 4.32 (operator precedence) should be noted. However, case e1 in the LR parser is frequently handled by the reduction processor of the operator-precedence parser.

- e1: /* This routine is called from states 0, 2, 4 and 5, all of which expect the beginning of an operand, either an *id* or a left parenthesis. Instead, an operator, + or *, or the end of the input was found. */
 push an imaginary *id* onto the stack and cover it with state 3 (the goto of states 0, 2, 4 and 5 on *id*)⁵
 issue diagnostic "missing operand"
- e2: /* This routine is called from states 0, 1, 2, 4 and 5 on finding a right parenthesis. */
 remove the right parenthesis from the input
 issue diagnostic "unbalanced right parenthesis"
- e3: /* This routine is called from states 1 or 6 when expecting an operator, and an *id* or right parenthesis is found. */
 push + onto the stack and cover it with state 4.
 issue diagnostic "missing operator"
- e4: /* This routine is called from state 6 when the end of the input is found.

⁵ Note that in practice grammar symbols are not placed on the stack. It is useful to imagine them there to remind us of the symbols which the states "represent."

State 6 expects an operator or a right parenthesis. */
 push a right parenthesis onto the stack and cover it with state 9.
 issue diagnostic "missing right parenthesis"

On the erroneous input `id + )` discussed in Example 4.32, the sequence of configurations entered by the parser is shown in Fig. 4.54. □

STACK	INPUT	ERROR MESSAGE AND ACTION
0	<code>id+)\$</code>	
<code>0id3</code>	<code>+)\$</code>	
<code>0E1</code>	<code>+)\$</code>	
<code>0E1+4</code>	<code>)\$</code>	
<code>0E1+4</code>	<code>\$</code>	"unbalanced right parenthesis" c2 removes right parenthesis
<code>0E1+4id3</code>	<code>\$</code>	"missing operand" c1 pushes <code>id 3</code> on stack
<code>0E1+4E7</code>	<code>\$</code>	
<code>0E1</code>	<code>\$</code>	

Fig. 4.54. Parsing and error recovery moves made by LR parser.

4.9 PARSER GENERATORS

This section shows how a parser generator can be used to facilitate the construction of the front end of a compiler. We shall use the LALR parser generator Yacc as the basis of our discussion, since it implements many of the concepts discussed in the previous two sections and it is widely available. Yacc stands for "yet another compiler-compiler," reflecting the popularity of parser generators in the early 1970's when the first version of Yacc was created by S. C. Johnson. Yacc is available as a command on the UNIX system, and has been used to help implement hundreds of compilers.

The Parser Generator Yacc

A translator can be constructed using Yacc in the manner illustrated in Fig. 4.55. First, a file, say `translate.y`, containing a Yacc specification of the translator is prepared. The UNIX system command

```
yacc translate.y
```

transforms the file `translate.y` into a C program called `y.tab.c` using the LALR method outlined in Algorithm 4.13. The program `y.tab.c` is a representation of an LALR parser written in C, along with other C routines that the user may have prepared. The LALR parsing table is compacted as described in Section 4.7. By compiling `y.tab.c` along with the `ly` library,

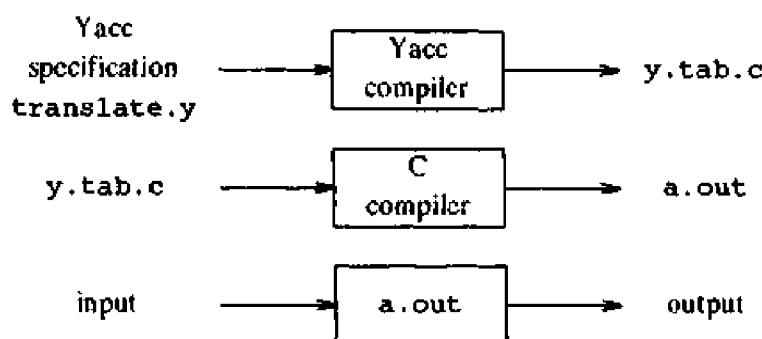


Fig. 4.55. Creating an input/output translator with Yacc.

that contains the LR parsing program using the command

```
cc y.tab.c -ly
```

we obtain the desired object program `a.out` that performs the translation specified by the original Yacc program.⁶ If other procedures are needed, they can be compiled or loaded with `y.tab.c`, just as with any C program.

A Yacc source program has three parts:

```

declarations
%%
translation rules
%%
supporting C-routines

```

Example 4.51. To illustrate how to prepare a Yacc source program, let us construct a simple desk calculator that reads an arithmetic expression, evaluates it, and then prints its numeric value. We shall build the desk calculator starting with the following grammar for arithmetic expressions:

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid \text{digit}
 \end{aligned}$$

The token **digit** is a single digit between 0 and 9. A Yacc desk calculator program derived from this grammar is shown in Fig. 4.56. $\square$

The declarations part. There are two optional sections in the declarations part of a Yacc program. In the first section, we put ordinary C declarations, delimited by `%{` and `%}`. Here we place declarations of any temporaries used by the translation rules or procedures of the second and third sections. In

⁶ The name `ly` is system dependent.


```

%{
#include <ctype.h>
%}

%token DIGIT

%%

line      :  expr '\n'          { printf("%d\n", $1); }
          ;
expr      :  expr '+' term      { $$ = $1 + $3; }
          |  term
          ;
term       :  term '*' factor   { $$ = $1 * $3; }
          |  factor
          ;
factor     :  '(' expr ')'      { $$ = $2; }
          |  DIGIT
          ;

%%

yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}

```

Fig. 4.56. Yacc specification of a simple desk calculator.

Fig. 4.56, this section contains only the include-statement

```
#include <ctype.h>
```

that causes the C preprocessor to include the standard header file `<ctype.h>` that contains the predicate `isdigit`.

Also in the declarations part are declarations of grammar tokens. In Fig. 4.56, the statement

```
%token DIGIT
```

declares `DIGIT` to be a token. Tokens declared in this section can then be used in the second and third parts of the Yacc specification.

The translation rules part. In the part of the Yacc specification after the first `%%` pair, we put the translation rules. Each rule consists of a grammar

production and the associated semantic action. A set of productions that we have been writing

$$\langle \text{left side} \rangle \rightarrow \langle \text{alt 1} \rangle \mid \langle \text{alt 2} \rangle \mid \cdots \mid \langle \text{alt } n \rangle$$

would be written in Yacc as

```

<left side>      :  <alt 1>      { semantic action 1 }
                  |  <alt 2>      { semantic action 2 }
                  . . .
                  |  <alt n>      { semantic action n }
                  ;

```

In a Yacc production, a quoted single character 'c' is taken to be the terminal symbol c, and unquoted strings of letters and digits not declared to be tokens are taken to be nonterminals. Alternative right sides can be separated by a vertical bar, and a semicolon follows each left side with its alternatives and their semantic actions. The first left side is taken to be the start symbol.

A Yacc semantic action is a sequence of C statements. In a semantic action, the symbol \$\$ refers to the attribute value associated with the nonterminal on the left, while \$i refers to the value associated with the *i*th grammar symbol (terminal or nonterminal) on the right. The semantic action is performed whenever we reduce by the associated production, so normally the semantic action computes a value for \$\$ in terms of the \$i's. In the Yacc specification, we have written the two *E*-productions

$$E \rightarrow E + T \mid T$$

and their associated semantic actions as

```

expr      :  expr '+' term      { $$ = $1 + $3; }
          |  term
          ;

```

Note that the nonterminal *term* in the first production is the third grammar symbol on the right, while '+' is the second. The semantic action associated with the first production adds the value of the *expr* and the *term* on the right and assigns the result as the value for the nonterminal *expr* on the left. We have omitted the semantic action for the second production altogether, since copying the value is the default action for productions with a single grammar symbol on the right. In general, { \$\$ = \$1; } is the default semantic action.

Notice that we have added a new starting production

```

line      :  expr '\n'          { printf("%d\n", $1); }

```

to the Yacc specification. This production says that an input to the desk calculator is to be an expression followed by a newline character. The semantic action associated with this production prints the decimal value of the expression followed by a newline character.

The supporting C-routines part. The third part of a Yacc specification consists of supporting C-routines. A lexical analyzer by the name `yylex()` must be provided. Other procedures such as error recovery routines may be added as necessary.

The lexical analyzer `yylex()` produces pairs consisting of a token and its associated attribute value. If a token such as `DIGIT` is returned, the token must be declared in the first section of the Yacc specification. The attribute value associated with a token is communicated to the parser through a Yacc-defined variable `yyval`.

The lexical analyzer in Fig. 4.56 is very crude. It reads input characters one at a time using the C-function `getchar()`. If the character is a digit, the value of the digit is stored in the variable `yyval`, and the token `DIGIT` is returned. Otherwise, the character itself is returned as the token.

Using Yacc with Ambiguous Grammars

Let us now modify the Yacc specification so that the resulting desk calculator becomes more useful. First, we shall allow the desk calculator to evaluate a sequence of expressions, one to a line. We shall also allow blank lines between expressions. We do this by changing the first rule to

```
lines : lines expr '\n' { printf("%g\n", $2); }
      | lines '\n'
      |
      ;
```

In Yacc, an empty alternative, as the third line is, denotes ϵ .

Second, we shall enlarge the class of expressions to include numbers instead of single digits and to include the arithmetic operators $+$, $-$ (both binary and unary), $*$, and $/$. The easiest way to specify this class of expressions is to use the ambiguous grammar

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid \sim E \mid \text{number}$$

The resulting Yacc specification is shown in Fig. 4.57.

Since the grammar in the Yacc specification in Fig. 4.57 is ambiguous, the LALR algorithm will generate parsing action conflicts. Yacc will report the number of parsing action conflicts that are generated. A description of the sets of items and the parsing action conflicts can be obtained by invoking Yacc with a `-v` option. This option generates an additional file `y.output` that contains the kernels of the sets of items found by the parsing, a description of the parsing action conflicts generated by the LALR algorithm, and a readable representation of the LR parsing table showing how the parsing action conflicts were resolved. Whenever Yacc reports that it has found parsing action conflicts, it is wise to create and consult the file `y.output` to see why the parsing action conflicts were generated and to see whether they were resolved correctly.

```

%{
#include <ctype.h>
#include <stdio.h>
#define YYSTYPE double /* double type for Yacc stack */
%}

%token NUMBER
%left '+' '-'
%left '*' '/'
%right UMINUS

%%
lines      : lines expr '\n' { printf("%g\n", $2); }
           | lines '\n'
           | /*  $\epsilon$  */
           ;
expr       : expr '+' expr { $$ = $1 + $3; }
           | expr '-' expr { $$ = $1 - $3; }
           | expr '*' expr { $$ = $1 * $3; }
           | expr '/' expr { $$ = $1 / $3; }
           | '(' expr ')' { $$ = $2; }
           | '-' expr %prec UMINUS { $$ = - $2; }
           | NUMBER
           ;

%%
yylex() {
    int c;
    while ( ( c = getchar() ) == ' ' );
    if ( ( c == '.' ) || ( isdigit(c) ) ) {
        ungetc(c, stdin);
        scanf("%lf", &yyval);
        return NUMBER;
    }
    return c;
}

```

Fig. 4.57. Yacc specification for a more advanced desk calculator.

Unless otherwise instructed Yacc will resolve all parsing action conflicts using the following two rules:

1. A reduce/reduce conflict is resolved by choosing the conflicting production listed first in the Yacc specification. Thus, in order to make the correct resolution in the typesetting grammar (4.25), it is sufficient to list production (1) ahead of production (3).

2. A shift/reduce conflict is resolved in favor of shift. This rule resolves the shift/reduce conflict arising from the dangling-else ambiguity correctly.

Since these default rules may not always be what the compiler writer wants, Yacc provides a general mechanism for resolving shift/reduce conflicts. In the declarations portion, we can assign precedences and associativities to terminals. The declaration

```
%left '+' '-'
```

makes + and - be of the same precedence and be left associative. We can declare an operator to be right associative by saying

```
%right '^'
```

and we can force an operator to be a nonassociative binary operator (i.e., two occurrences of the operator cannot be combined at all) by saying

```
%nonassoc '<'
```

The tokens are given precedences in the order in which they appear in the declarations part, lowest first. Tokens in the same declaration have the same precedence. Thus, the declaration

```
%right UMINUS
```

in Fig. 4.57 gives the token UMINUS a precedence level higher than that of the five preceding terminals.

Yacc resolves shift/reduce conflicts by attaching a precedence and associativity to each production involved in a conflict, as well as to each terminal involved in a conflict. If it must choose between shifting input symbol a and reducing by production $A \rightarrow \alpha$, Yacc reduces if the precedence of the production is greater than that of a , or if the precedences are the same and the associativity of the production is *left*. Otherwise, shift is the chosen action.

Normally, the precedence of a production is taken to be the same as that of its rightmost terminal. This is the sensible decision in most cases. For example, given productions

$$E \rightarrow E + E \mid E * E$$

we would prefer to reduce by $E \rightarrow E + E$ with lookahead +, because the + in the right side has the same precedence as the lookahead, but is left associative. With lookahead *, we would prefer to shift, because the lookahead has higher precedence than the + in the production.

In those situations where the rightmost terminal does not supply the proper precedence to a production, we can force a precedence by appending to a production the tag

```
%prec <terminal>
```

The precedence and associativity of the production will then be the same as that of the terminal, which presumably is defined in the declaration section.

Yacc does not report shift/reduce conflicts that are resolved using this precedence and associativity mechanism.

This "terminal" can be a placeholder, like `UMINUS` in Fig. 4.57; this terminal is not returned by the lexical analyzer, but is declared solely to define a precedence for a production. In Fig. 4.57, the declaration

```
%right UMINUS
```

assigns to the token `UMINUS` a precedence that is higher than that of `*` and `/`. In the translation rules part, the tag

```
%prec UMINUS
```

at the end of the production

```
expr      :  '-' expr
```

makes the unary minus operator in this production have a higher precedence than any other operator.

Creating Yacc Lexical Analyzers with Lex

Lex was designed to produce lexical analyzers that could be used with Yacc. The Lex library 11 will provide a driver program named `yylex()`, the name required by Yacc for its lexical analyzer. If Lex is used to produce the lexical analyzer, we replace the routine `yylex()` in the third part of the Yacc specification by the statement

```
#include "lex.yy.c"
```

and we have each Lex action return a terminal known to Yacc. By using the `#include "lex.yy.c"` statement, the program `yylex` has access to Yacc's names for tokens, since the Lex output file is compiled as part of the Yacc output file `y.tab.c`.

Under the UNIX system, if the Lex specification is in the file `first.l` and the Yacc specification in `second.y`, we can say

```
lex first.l
yacc second.y
cc y.tab.c -ly -ll
```

to obtain the desired translator.

The Lex specification in Fig. 4.58 can be used in place of the lexical analyzer in Fig. 4.57. The last pattern is `\n!`, since `.` in Lex matches any character but a newline.

Error Recovery in Yacc

In Yacc, error recovery can be performed using a form of error productions. First, the user decides what "major" nonterminals will have error recovery associated with them. Typical choices are some subset of the nonterminals

```

number      [0-9]+\.[0-9]*| [0-9]+\.[0-9]+
%%
[ ]         { /* skip blanks */ }
{number}    { sscanf(yytext, "%lf", &yyval);
              return NUMBER; }
\nl.       { return yytext[0]; }

```

Fig. 4.58. Lex specification for `yyllex()` in Fig. 4.57.

generating expressions, statements, blocks, and procedures. The user then adds to the grammar error productions of the form $A \rightarrow \text{error } \alpha$, where A is a major nonterminal and α is a string of grammar symbols, perhaps the empty string; **error** is a Yacc reserved word. Yacc will generate a parser from such a specification, treating the error productions as ordinary productions.

However, when the parser generated by Yacc encounters an error, it treats the states whose sets of items contain error productions in a special way. On encountering an error, Yacc pops symbols from its stack until it finds the top-most state on its stack whose underlying set of items includes an item of the form $A \rightarrow \cdot \text{error } \alpha$. The parser then “shifts” a fictitious token **error** onto the stack, as though it saw the token **error** on its input.

When α is ϵ , a reduction to A occurs immediately and the semantic action associated with the production $A \rightarrow \text{error}$ (which might be a user-specified error-recovery routine) is invoked. The parser then discards input symbols until it finds an input symbol on which normal parsing can proceed.

If α is not empty, Yacc skips ahead on the input looking for a substring that can be reduced to α . If α consists entirely of terminals, then it looks for this string of terminals on the input, and “reduces” them by shifting them onto the stack. At this point, the parser will have **error** α on top of its stack. The parser will then reduce **error** α to A , and resume normal parsing.

For example, an error production of the form

stmt $\rightarrow$ **error** ;

would specify to the parser that it should skip just beyond the next semicolon on seeing an error, and assume that a statement had been found. The semantic routine for this error production would not need to manipulate the input, but could generate a diagnostic message and set a flag to inhibit generation of object code, for example.

Example 4.52. Figure 4.59 shows the Yacc desk calculator of Fig. 4.57 with the error production

lines : **error** '\n'

This error production causes the desk calculator to suspend normal parsing when a syntax error is found on an input line. On encountering the error, the

```

%{
#include <ctype.h>
#include <stdio.h>
#define YYSTYPE double /* double type for Yacc stack */
%}

%token NUMBER
%left '+' '-'
%left '*' '/'
%right UMINUS

%%
lines    : lines expr '\n' { printf("%g\n", $2); }
        | lines '\n'
        | /* empty */
        | error '\n' { yyerror("reenter last line:");
                      yyerrok; }
;

expr     : expr '+' expr { $$ = $1 + $3; }
        | expr '-' expr { $$ = $1 - $3; }
        | expr '*' expr { $$ = $1 * $3; }
        | expr '/' expr { $$ = $1 / $3; }
        | '(' expr ')' { $$ = $2; }
        | '-' expr %prec UMINUS { $$ = - $2; }
        | NUMBER
;

%%
#include "lex.yy.c"

```

Fig. 4.59. Desk calculator with error recovery.

parser in the desk calculator starts popping symbols from its stack until it encounters a state that has a shift action on the token **error**. State 0 is such a state (in this example, it's the only such state), since its items include

lines → · **error** '\n'

Also, state 0 is always on the bottom of the stack. The parser shifts the token **error** onto the stack, and then proceeds to skip ahead in the input until it has found a newline character. At this point the parser shifts the newline onto the stack, reduces **error** '\n' to *lines*, and emits the diagnostic message "reenter last line:". The special Yacc routine **yyerrok** resets the parser to its normal mode of operation. □

EXERCISES

4.1 Consider the grammar

$$\begin{aligned} S &\rightarrow (L) \mid a \\ L &\rightarrow L , S \mid S \end{aligned}$$

- What are the terminals, nonterminals, and start symbol?
- Find parse trees for the following sentences:
 - (a, a)
 - $(a, (a, a))$
 - $(a, ((a, a), (a, a)))$
- Construct a leftmost derivation for each of the sentences in (b).
- Construct a rightmost derivation for each of the sentences in (b).
- *e) What language does this grammar generate?

4.2 Consider the grammar

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

- Show that this grammar is ambiguous by constructing two different leftmost derivations for the sentence $abab$.
- Construct the corresponding rightmost derivations for $abab$.
- Construct the corresponding parse trees for $abab$.
- *d) What language does this grammar generate?

4.3 Consider the grammar

$$\begin{aligned} bexpr &\rightarrow bexpr \text{ or } bterm \mid bterm \\ bterm &\rightarrow bterm \text{ and } bfactor \mid bfactor \\ bfactor &\rightarrow \text{not } bfactor \mid (bexpr) \mid \text{true} \mid \text{false} \end{aligned}$$

- Construct a parse tree for the sentence **not (true or false)**.
- Show that this grammar generates all boolean expressions.
- *c) Is this grammar ambiguous? Why?

4.4 Consider the grammar

$$R \rightarrow R' \mid R \mid RR \mid R^* \mid (R) \mid a \mid b$$

Note that the first vertical bar is the “or” symbol, not a separator between alternatives.

- Show that this grammar generates all regular expressions over the symbols a and b .
- Show that this grammar is ambiguous.
- *c) Construct an equivalent unambiguous grammar that gives the operators $*$, concatenation, and $|$ the precedences and associativities defined in Section 3.3.
- d) Construct a parse tree in both grammars for the sentence $a|b^*c$.

- 4.5 The following grammar for **if-then-else** statements is proposed to remedy the dangling-else ambiguity:

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \\ &\quad | \text{ matched_stmt} \\ matched_stmt &\rightarrow \text{if } expr \text{ then } matched_stmt \text{ else } stmt \\ &\quad | \text{ other} \end{aligned}$$

Show that this grammar is still ambiguous.

- *4.6 Try to design a grammar for each of the following languages. Which languages are regular?
- The set of all strings of 0's and 1's such that every 0 is immediately followed by at least one 1.
 - Strings of 0's and 1's with an equal number of 0's and 1's.
 - Strings of 0's and 1's with an unequal number of 0's and 1's.
 - Strings of 0's and 1's in which 011 does not appear as a substring.
 - Strings of 0's and 1's of the form xy where $x \neq y$.
 - Strings of 0's and 1's of the form xx .
- 4.7 Construct a grammar for the expressions of each of the following languages.
- Pascal
 - C
 - Fortran 77
 - Ada
 - Lisp
- 4.8 Construct unambiguous grammars for the statements in each of the languages of Exercise 4.7.
- 4.9 We can use regular-expression-like operators in the right sides of grammar productions. Square brackets can be used to denote an optional part of a production. For example, we might write

$$stmt \rightarrow \text{if } expr \text{ then } stmt \mid \text{ else } stmt \mid$$

to denote an optional else-statement. In general, $A \rightarrow \alpha [\beta] \gamma$ is equivalent to the two productions $A \rightarrow \alpha\beta\gamma$ and $A \rightarrow \alpha\gamma$.

Curly brackets can be used to denote a phrase that can be repeated zero or more times. For example,

$$stmt \rightarrow \text{begin } stmt \{ ; stmt \} \text{ end}$$

denotes a list of semicolon-separated *stmt*'s enclosed between **begin** and **end**. In general, $A \rightarrow \alpha \{ \beta \} \gamma$ is equivalent to $A \rightarrow \alpha B \gamma$ and $B \rightarrow \beta B \mid \epsilon$.

In a sense, $[\beta]$ stands for the regular expression $\beta \mid \epsilon$, and $\{ \beta \}$ stands for β^* . We may generalize these notations to allow any

regular expression of grammar symbols on the right side of productions.

- Modify the above *stmt*-production so that a semicolon-terminated list of *stmt*'s appears on the right side.
- Give a set of context-free productions generating the same set of strings as $A \rightarrow B^*a(C|D)$.
- Show how to replace any production $A \rightarrow r$, where r is a regular expression, by a finite collection of context-free productions.

4.10 The following grammar generates declarations for a single identifier:

```

    stmt → declare id option_list
    option_list → option_list option | ε
    option → mode | scale | precision | base
    mode → real | complex
    scale → fixed | floating
    precision → single | double
    base → binary | decimal
  
```

- Show how this grammar can be generalized to permit n options A_i , $1 \leq i \leq n$, each of which can be either a_i or b_i .
- The above grammar permits redundant or contradictory declarations such as

```
declare zap real fixed real floating
```

We could insist that the syntax of the language forbid such declarations. We are thus left with a finite number of token sequences that are syntactically correct. Obviously these legal declarations form a context-free language, indeed a regular set. Write a grammar for declarations with n options, each option appearing at most once.

- **c) Show that a grammar for part (b) must have at least 2^n symbols.
- d) What does (c) say about the feasibility of enforcing nonredundancy and noncontradiction among options in declarations via the syntactic definition of a language?

- Eliminate the left-recursion from the grammar in Exercise 4.1.
- Construct a predictive parser for the grammar in (a). Show the behavior of the parser on the sentences in Exercise 4.1(b).

4.12 Construct a recursive-descent parser with backtracking for the grammar in Exercise 4.2. Can you construct a predictive parser for this grammar?

4.13 The grammar

$$S \rightarrow aSa \mid aa$$

generates all even length strings of a 's except for the empty string.

- a) Construct a recursive-descent parser with backtracking for this grammar that tries the alternative aSa before aa . Show that the procedure for S succeeds on 2, 4, or 8 a 's, but fails on 6 a 's.
- *b) What language does your parser recognize?

4.14 Construct a predictive parser for the grammar in Exercise 4.3.

4.15 Construct a predictive parser from the unambiguous grammar for regular expressions in Exercise 4.4.

***4.16** Show that no left-recursive grammar can be LL(1).

***4.17** Show that no LL(1) grammar can be ambiguous.

4.18 Show that a grammar with no ϵ -productions in which each alternative begins with a distinct terminal is always LL(1).

4.19 A grammar symbol X is *useless* if there is no derivation of the form $S \xRightarrow{*} wXy \xRightarrow{*} wxy$. That is, X can never appear in the derivation of some sentence.

- *a) Write an algorithm to eliminate all productions containing useless symbols from a grammar.
- b) Apply your algorithm to the grammar

$$\begin{aligned} S &\rightarrow 0 \mid A \\ A &\rightarrow AB \\ B &\rightarrow 1 \end{aligned}$$

4.20 We say a grammar is ϵ -free if either it has no ϵ -productions or there is exactly one ϵ -production $S \rightarrow \epsilon$ and then the start symbol S does not appear on the right side of any production.

- a) Write an algorithm to convert a given grammar into an equivalent ϵ -free grammar. *Hint.* First determine all nonterminals that can generate the empty string.
- b) Apply your algorithm to the grammar in Exercise 4.2.

4.21 A *single* production is one with a single nonterminal as its right side.

- a) Write an algorithm to convert a grammar into an equivalent grammar with no single productions.
- b) Apply your algorithm to the expression grammar (4.10).

4.22 A *cycle-free* grammar has no derivations of the form $A \xRightarrow{+} A$ for any nonterminal A .

- a) Write an algorithm to convert a grammar into an equivalent cycle-free grammar.
- b) Apply your algorithm to the grammar $S \rightarrow SS \mid (S) \mid \epsilon$.

4.23 a) Using the grammar in Exercise 4.1 construct a rightmost derivation for $(a, (a, a))$ and show the handle of each right-sentential form.

- b) Show the steps of a shift-reduce parser corresponding to the right-most derivation of (a).
 c) Show the steps in the bottom-up construction of a parse tree during the shift-reduce parse of (b).

4.24 Figure 4.60 shows operator-precedence relations for the grammar of Exercise 4.1. Using these precedence relations, parse the sentences in Exercise 4.1(b).

	a	$($	$)$	$,$	$\$$
a			$\cdot >$	$\cdot >$	$\cdot >$
$($	$< \cdot$	$< \cdot$	$=$	$< \cdot$	
$)$			$\cdot >$	$\cdot >$	$\cdot >$
$,$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$	
$\$$	$< \cdot$	$< \cdot$			

Fig. 4.60. Operator-precedence relations for the grammar of Exercise 4.1.

- 4.25** Find operator-precedence functions for the table of Fig. 4.60.
- 4.26** There is a mechanical way to produce operator-precedence relations from an operator grammar, including those with many different non-terminals. Define *leading*(A) for nonterminal A to be the set of terminals a such that a is the leftmost terminal in some string derived from A , and define *trailing*(A) to be the set of terminals that can be the rightmost in a string derived from A . Then for terminals a and b , we say $a \doteq b$ if there is a right side of the form $\alpha a \beta b \gamma$, where β is either empty or a single nonterminal, and α and γ are arbitrary. We say $a < \cdot b$ if there is a right side of the form $\alpha a A \beta$, and b is in *leading*(A); we say $a \cdot > b$ if there is a right side of the form $\alpha A b \beta$, and a is in *trailing*(A). In both cases, α and β are arbitrary strings. Also, $\$ < \cdot b$ whenever b is in *leading*(S), where S is the start symbol, and $a \cdot > \$$ whenever a is in *trailing*(S).
- a) For the grammar of Exercise 4.1 compute *leading* and *trailing* for S and T .
- b) Verify that the precedence relations of Figure 4.60 are the ones derived from this grammar.
- 4.27** Generate operator-precedence relations for the following grammars.
- a) The grammar of Exercise 4.2.
- b) The grammar of Exercise 4.3.
- c) The expression grammar (4.10).
- 4.28** Construct an operator-precedence parser for regular expressions.

- 4.29** A grammar is said to be a (uniquely invertible) *operator-precedence grammar* if it is an operator grammar with no two right sides that have the same pattern of terminals, and the method of Exercise 4.26 yields at most one precedence relation between any pair of terminals. Which of the grammars of Exercise 4.27 are operator-precedence grammars?
- 4.30** A grammar is said to be in *Greibach normal form* (GNF) if it is ϵ -free and each production (except $S \rightarrow \epsilon$ if it exists) is of the form $A \rightarrow a\alpha$, where a is a terminal and α is a string of nonterminals, possibly empty.
- **a)** Write an algorithm to convert a grammar into an equivalent Greibach normal form grammar.
- b)** Apply your algorithm to the expression grammar (4.10).
- *4.31** Show that every grammar can be converted into an equivalent operator grammar. *Hint.* First transform the grammar into Greibach normal form.
- *4.32** Show that every grammar can be converted into an operator grammar in which each production is of one of the forms

$$A \rightarrow aBcC \quad A \rightarrow aBb \quad A \rightarrow aB \quad A \rightarrow a$$

If ϵ is in the language, then $S \rightarrow \epsilon$ is also a production.

- 4.33** Consider the ambiguous grammar

$$\begin{array}{l} S \rightarrow AS \mid b \\ A \rightarrow SA \mid a \end{array}$$

- Construct the collection of sets of LR(0) items for this grammar.
 - Construct an NFA in which each state is an LR(0) item from (a). Show that the goto graph of the canonical collection of LR(0) items for this grammar is the same as the DFA constructed from the NFA using the subset construction.
 - Construct the parsing table using the SLR Algorithm 4.8.
 - Show all moves allowed by the table from (c) on the input *abab*.
 - Construct the canonical parsing table.
 - Construct the parsing table using the LALR Algorithm 4.11.
 - Construct the parsing table using the LALR Algorithm 4.13.
- 4.34** Construct an SLR parsing table for the grammar in Exercise 4.3.
- 4.35** Consider the following grammar

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T F \mid F \\ F \rightarrow F^* \mid a \mid b \end{array}$$

- a) Construct the SLR parsing table for this grammar.
- b) Construct the LALR parsing table.

4.36 Compact the parsing tables constructed in Exercises 4.33, 4.34, and 4.35, according to the method of Section 4.7.

4.37 a) Show that the following grammar

$$\begin{aligned} S &\rightarrow AaAb \mid BbBa \\ A &\rightarrow \epsilon \\ B &\rightarrow \epsilon \end{aligned}$$

is LL(1) but not SLR(1).

****b)** Show that every LL(1) grammar is an LR(1) grammar.

***4.38** Show that no LR(1) grammar can be ambiguous.

4.39 Show that the following grammar

$$\begin{aligned} S &\rightarrow Aa \mid bAc \mid dc \mid bda \\ A &\rightarrow d \end{aligned}$$

is LALR(1) but not SLR(1).

4.40 Show that the following grammar

$$\begin{aligned} S &\rightarrow Aa \mid bAc \mid Bc \mid bBa \\ A &\rightarrow d \\ B &\rightarrow d \end{aligned}$$

is LR(1) but not LALR(1).

***4.41** Consider the family of grammars G_n defined by:

$$\begin{aligned} S &\rightarrow A_i b_i & 1 \leq i \leq n \\ A_i &\rightarrow a_j A_i \mid a_j & 1 \leq i, j \leq n \text{ and } j \neq i \end{aligned}$$

- a) Show that G_n has $2n^2 - n$ productions and $2^n + n^2 + n$ sets of LR(0) items. What does this result say about how big an LR parser can get compared to the size of the grammar?
- b) Is G_n SLR(1)?
- c) Is G_n LALR(1)?

4.42 Write an algorithm to compute for each nonterminal A in a grammar the set of nonterminals B such that $A \xRightarrow{*} B\alpha$ for some string of grammar symbols α .

4.43 Write an algorithm to compute for each nonterminal A in a grammar the set of terminals a such that $A \xRightarrow{*} aw$ for some string of terminals w , where the last step of the derivation does not use an ϵ -production.

4.44 Construct an SLR parsing table for the grammar of Exercise 4.4. Resolve the parsing action conflicts in such a way that regular expressions will be parsed normally.

4.45 Construct an SLR parser for the dangling-else grammar (4.7), treating *expr* as a terminal. Resolve the parsing action conflict in the usual way.

4.46 a) Construct an SLR parsing table for the grammar

$$\begin{aligned} E &\rightarrow E \text{ sub } R \mid E \text{ sup } E \mid \{ E \} \mid c \\ R &\rightarrow E \text{ sup } E \mid E \end{aligned}$$

Resolve the parsing action conflict so that expressions will be parsed in the same way as by the LR parser in Fig. 4.52.

b) Can every reduce/reduce conflict generated in the LR parsing table construction process be converted into a shift/reduce conflict by transforming the grammar?

***4.47** Construct an equivalent LR grammar for the typesetting grammar (4.25) that factors out expressions of the form $E \text{ sub } E \text{ sup } E$ as a special case.

***4.48** Consider the following ambiguous grammar for n binary infix operators:

$$E \rightarrow E \theta_1 E \mid E \theta_2 E \mid \cdots \mid E \theta_n E \mid (E) \mid \text{id}$$

Assume that all operators are left-associative and that θ_i takes precedence over θ_j if $i > j$.

a) Construct the SLR sets of items for this grammar. How many sets of items are there, as a function of n ?

b) Construct the SLR parsing table for this grammar and compact it using the list representation in Section 4.7. What is the total length of all the lists used in the representation, as a function of n ?

c) How many steps does it take to parse $\text{id } \theta_i \text{ id } \theta_j \text{ id}$?

***4.49** Repeat Exercise 4.48 for the unambiguous grammar

$$\begin{aligned} E_1 &\rightarrow E_1 \theta_1 E_2 \mid E_2 \\ E_2 &\rightarrow E_2 \theta_2 E_3 \mid E_3 \\ &\vdots \\ E_n &\rightarrow E_n \theta_n E_{n+1} \mid E_{n+1} \\ E_{n+1} &\rightarrow (E_1) \mid \text{id} \end{aligned}$$

What do your answers to Exercises 4.48 and 4.49 say about the relative efficiency of parsers for equivalent ambiguous and unambiguous grammars? What about the relative efficiency of constructing the parser?

4.50 Write a Yacc program that will take arithmetic expressions as input and produce the corresponding postfix expression as output.

- 4.51** Write a Yacc “desk calculator” program that will evaluate boolean expressions.
- 4.52** Write a Yacc program that will take a regular expression as input and produce its parse tree as output.
- 4.53** Trace out the moves that would be made by the predictive, operator-precedence, and LR parsers of Examples 4.20, 4.32, and 4.50 on the following erroneous inputs:
- $(id + (* id))$
 - $* + id) + (id *$
- *4.54** Construct error-correcting operator-precedence and LR parsers for the following grammar:

$$\begin{array}{lcl}
 stmt & \rightarrow & \text{if } e \text{ then } stmt \\
 & | & \text{if } e \text{ then } stmt \text{ else } stmt \\
 & | & \text{while } e \text{ do } stmt \\
 & | & \text{begin } list \text{ end} \\
 & | & s \\
 list & \rightarrow & list ; stmt \\
 & | & stmt
 \end{array}$$

- *4.55** The grammar in Exercise 4.54 can be made LL by replacing the productions for *list* by

$$\begin{array}{lcl}
 list & \rightarrow & stmt list' \\
 list' & \rightarrow & ; stmt \mid \epsilon
 \end{array}$$

Construct an error-correcting predictive parser for the revised grammar.

- 4.56** Show the behavior of your parsers of Exercises 4.54 and 4.55 on the erroneous inputs
- $\text{if } e \text{ then } s ; \text{if } e \text{ then } s \text{ end}$
 - $\text{while } e \text{ do begin } s ; \text{if } e \text{ then } s ; \text{end}$
- 4.57** Write predictive, operator-precedence, and LR parsers with panic-mode error recovery for the grammars of Exercises 4.54 and 4.55, using semicolon and **end** as synchronizing tokens. Show the behavior of your parsers on the erroneous inputs of Exercise 4.56.
- 4.58** In Section 4.6, we proposed a graph-oriented method for determining the set of strings that could be popped from the stack in a reduce move of an operator-precedence parser.
- Give an algorithm for finding a regular expression denoting all such strings.
 - Give an algorithm to determine whether the set of such strings is finite or infinite, listing them if finite.

c) Apply your algorithms from (a) and (b) to the grammar of Exercise 4.54.

****4.59** We made the claim for the error-correcting parsers of Figs. 4.18, 4.28, and 4.53 that any error correction eventually resulted in at least one more symbol being removed from the input or the stack being shortened if the end of the input has been reached. The corrections chosen, however, did not all cause an input symbol to be consumed immediately. Can you prove that no infinite loops are possible for the parsers of Figs. 4.18, 4.28, and 4.53? *Hint.* It helps to observe that for the operator-precedence parser, consecutive terminals on the stack are related by $\leq$, even if there have been errors. For the LR parser, the stack will still contain a viable prefix, even in the presence of errors.

****4.60** Give an algorithm for detecting unreachable entries in predictive, operator-precedence, and LR parsing tables.

4.61 The LR parser of Fig. 4.53 handles the four situations in which the top state is 4 or 5 (which occur when + and * are on top of the stack, respectively) and the next input is + or * in exactly the same way: by calling the routine *el*, which inserts an *id* between them. We could easily envision an LR parser for expressions involving the full set of arithmetic operators behaving in the same fashion: insert *id* between the adjacent operators. In certain languages (such as PL/I or C but not Fortran or Pascal), it would be wise to treat, in a special way, the case in which / is on top of the stack and * is the next input. Why? What would be a reasonable course of action for the error corrector to take?

4.62 A grammar is said to be in *Chomsky normal form* (CNF) if it is ϵ -free and each non- ϵ -production is of the form $A \rightarrow BC$ or of the form $A \rightarrow a$.

*a) Give an algorithm to convert a grammar into an equivalent Chomsky normal form grammar.

b) Apply your algorithm to the expression grammar (4.10).

4.63 Given a grammar G in Chomsky normal form and an input string $w = a_1 a_2 \cdots a_n$, write an algorithm to determine whether w is in $L(G)$. *Hint.* Using dynamic programming fill in an $n \times n$ table T in which $T[i, j] = \{A \mid A \xRightarrow{*} a_i a_{i+1} \cdots a_j\}$. The input string w is in $L(G)$ if and only if S is in $T[1, n]$.

***4.64** a) Given a Chomsky normal form grammar G , show how to add productions for single insertion, deletion, and mutation errors to the grammar so that the enlarged grammar generates all possible token strings.

- b) Modify the parsing algorithm in Exercise 4.63 so that, given any string w , it will find a parse for w that uses the fewest number of error productions.

4.65 Write a Yacc parser for arithmetic expressions that uses the error-recovery mechanism in Example 4.50.

BIBLIOGRAPHIC NOTES

The highly influential Algol 60 report (Naur [1963]) used Backus-Naur Form (BNF) to define the syntax of a major programming language. The equivalence of BNF and context-free grammars was quickly noted, and the theory of formal languages received a great deal of attention in the 1960's. Hopcroft and Ullman [1979] cover the basics of the field.

Parsing methods became much more systematic after the development of context-free grammars. Several general techniques for parsing any context-free grammar were invented. One of the earliest is the dynamic programming technique suggested in Exercise 4.63, which was independently discovered by J. Cocke, Younger [1967], and Kasami [1965]. As his Ph. D. thesis, Earley [1970] also developed a universal parsing algorithm for all context-free grammars. Aho and Ullman [1972b and 1973a] discuss these and other parsing methods in detail.

Many different parsing methods have been employed in compilers. Sheridan [1959] describes the parsing method used in the original Fortran compiler that introduced additional parentheses around operands in order to be able to parse expressions. The idea of operator precedence and the use of precedence functions is from Floyd [1963]. In the 1960's, a large number of bottom-up parsing strategies were proposed. These include simple precedence (Wirth and Weber [1966]), bounded-context (Floyd [1964], Graham [1964]), mixed-strategy precedence (McKeeman, Horning, and Wortman [1970]), and weak precedence (Ichbiah and Morse [1970]).

Recursive-descent and predictive parsing are widely used in practice. Because of its flexibility, recursive-descent parsing was used in many early compiler-writing systems such as META (Schorre [1964]) and TMG (McClure [1965]). A solution to Exercise 4.13 can be found in Birman and Ullman [1973], along with some of the theory of this parsing method. Pratt [1973] proposes a top-down operator-precedence parsing method.

LL grammars were studied by Lewis and Stearns [1968] and their properties were developed in Rosenkrantz and Stearns [1970]. Predictive parsers were studied extensively by Knuth [1971a]. Lewis, Rosenkrantz, and Stearns [1976] describe the use of predictive parsers in compilers. Algorithms for transforming grammars into LL(1) form are presented in Foster [1968], Wood [1969], Stearns [1971], and Soisalon-Soininen and Ukkonen [1979].

LR grammars and parsers were first introduced by Knuth [1965] who described the construction of canonical LR parsing tables. The LR method was not deemed practical until Korenjak [1969] showed that with it

reasonable-sized parsers could be produced for programming language grammars. When DeRemer [1969, 1971] devised the SLR and LALR methods, which are simpler than Korenjak's, the LR technique became the method of choice for automatic parser generators. Today, LR parser generators are common in compiler-construction environments.

A great deal of research went into the engineering of LR parsers. The use of ambiguous grammars in LR parsing is due to Aho, Johnson, and Ullman [1975] and Earley [1975a]. The elimination of reductions by single productions has been discussed in Anderson, Eve, and Horning [1973], Aho and Ullman [1973b], Demers [1975], Backhouse [1976], Joliat [1976], Pager [1977b], Soisalon-Soininen [1980], and Tokuda [1981].

Techniques for computing LALR(1) lookahead sets have been proposed by LaLonde [1971], Anderson, Eve, and Horning [1973], Pager [1977a], Kristensen and Madsen [1981], DeRemer and Pennello [1982], and Park, Choe, and Chang [1985] who also provide some experimental comparisons.

Aho and Johnson [1974] give a general survey of LR parsing and discuss some of the algorithms underlying the Yacc parser generator, including the use of error productions for error recovery. Aho and Ullman [1972b and 1973a] give an extensive treatment of LR parsing and its theoretical underpinnings.

Many error-recovery techniques for parsers have been proposed. Error-recovery techniques are surveyed by Ciesinger [1979] and by Sippu [1981]. Irons [1963] proposed a grammar-based approach to syntactic error recovery. Error productions were employed by Wirth [1968] for handling errors in a PL360 compiler. Leinius [1970] proposed the strategy of phrase-level recovery. Aho and Peterson [1972] show how global least-cost error recovery can be achieved using error productions in conjunction with general parsing algorithms for context-free grammars. Mauney and Fischer [1982] extend these ideas to local least-cost repair for LL and LR parsers using the parsing technique of Graham, Harrison, and Ruzzo [1980]. Graham and Rhodes [1975] discuss error recovery in the context of precedence parsing.

Horning [1976] discusses qualities good error messages should have. Sippu and Soisalon-Soininen [1983] compare the performance of the error-recovery technique in the Helsinki Language Processor (Räihä et al. [1983]) with the "forward move" recovery technique of Pennello and DeRemer [1978], the LR error-recovery technique of Graham, Haley, and Joy [1979], and the "global context" recovery technique of Pai and Kiebert [1980].

Error correction during parsing is discussed by Conway and Maxwell [1963], Moulton and Muller [1967], Conway and Wilcox [1973], Levy [1975], Tai [1978], and Röhrich [1980]. Aho and Peterson [1972] contains a solution to Exercise 4.63.

Syntax-Directed Translation

This chapter develops the theme of Section 2.3, the translation of languages guided by context-free grammars. We associate information with a programming language construct by attaching attributes to the grammar symbols representing the construct. Values for attributes are computed by “semantic rules” associated with the grammar productions.

There are two notations for associating semantic rules with productions, syntax-directed definitions and translation schemes. Syntax-directed definitions are high-level specifications for translations. They hide many implementation details and free the user from having to specify explicitly the order in which translation takes place. Translation schemes indicate the order in which semantic rules are to be evaluated, so they allow some implementation details to be shown. We use both notations in Chapter 6 for specifying semantic checking, particularly the determination of types, and in Chapter 8 for generating intermediate code.

Conceptually, with both syntax-directed definitions and translation schemes, we parse the input token stream, build the parse tree, and then traverse the tree as needed to evaluate the semantic rules at the parse-tree nodes (see Fig. 5.1). Evaluation of the semantic rules may generate code, save information in a symbol table, issue error messages, or perform any other activities. The translation of the token stream is the result obtained by evaluating the semantic rules.



Fig. 5.1. Conceptual view of syntax-directed translation.

An implementation does not have to follow the outline in Fig. 5.1 literally. Special cases of syntax-directed definitions can be implemented in a single pass by evaluating semantic rules during parsing, without explicitly constructing a parse tree or a graph showing dependencies between attributes. Since single-pass implementation is important for compile-time efficiency, much of this

chapter is devoted to studying such special cases. One important subclass, called the "L-attributed" definitions, encompasses virtually all translations that can be performed without explicit construction of a parse tree.

5.1 SYNTAX-DIRECTED DEFINITIONS

A syntax-directed definition is a generalization of a context-free grammar in which each grammar symbol has an associated set of attributes, partitioned into two subsets called the synthesized and inherited attributes of that grammar symbol. If we think of a node for the grammar symbol in a parse tree as a record with fields for holding information, then an attribute corresponds to the name of a field.

An attribute can represent anything we choose: a string, a number, a type, a memory location, or whatever. The value of an attribute at a parse-tree node is defined by a semantic rule associated with the production used at that node. The value of a synthesized attribute at a node is computed from the values of attributes at the children of that node in the parse tree; the value of an inherited attribute is computed from the values of attributes at the siblings and parent of that node.

Semantic rules set up dependencies between attributes that will be represented by a graph. From the dependency graph, we derive an evaluation order for the semantic rules. Evaluation of the semantic rules defines the values of the attributes at the nodes in the parse tree for the input string. A semantic rule may also have side effects, e.g., printing a value or updating a global variable. Of course, an implementation need not explicitly construct a parse tree or a dependency graph; it just has to produce the same output for each input string.

A parse tree showing the values of attributes at each node is called an annotated parse tree. The process of computing the attribute values at the nodes is called *annotating* or *decorating* the parse tree.

Form of a Syntax-Directed Definition

In a syntax-directed definition, each grammar production $A \rightarrow \alpha$ has associated with it a set of semantic rules of the form $b := f(c_1, c_2, \dots, c_k)$ where f is a function, and either

1. b is a synthesized attribute of A and $c_1, c_2, \dots, c_k$ are attributes belonging to the grammar symbols of the production, or
2. b is an inherited attribute of one of the grammar symbols on the right side of the production, and $c_1, c_2, \dots, c_k$ are attributes belonging to the grammar symbols of the production.

In either case, we say that attribute b *depends* on attributes $c_1, c_2, \dots, c_k$. An *attribute grammar* is a syntax-directed definition in which the functions in semantic rules cannot have side effects.

Functions in semantic rules will often be written as expressions. Occasionally, the only purpose of a semantic rule in a syntax-directed definition is to create a side effect. Such semantic rules are written as procedure calls or program fragments. They can be thought of as rules defining the values of dummy synthesized attributes of the nonterminal on the left side of the associated production; the dummy attribute and the $:=$ sign in the semantic rule are not shown.

Example 5.1. The syntax-directed definition in Fig. 5.2 is for a desk-calculator program. This definition associates an integer-valued synthesized attribute called *val* with each of the nonterminals *E*, *T*, and *F*. For each *E*, *T*, and *F*-production, the semantic rule computes the value of attribute *val* for the nonterminal on the left side from the values of *val* for the nonterminals on the right side.

PRODUCTION	SEMANTIC RULES
$L \rightarrow E \text{ n}$	<i>print</i> (<i>E.val</i>)
$E \rightarrow E_1 + T$	$E.val := E_1.val + T.val$
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T_1 * F$	$T.val := T_1.val \times F.val$
$T \rightarrow F$	$T.val := F.val$
$F \rightarrow (E)$	$F.val := E.val$
$F \rightarrow \text{digit}$	$F.val := \text{digit.lexval}$

Fig. 5.2. Syntax-directed definition of a simple desk calculator.

The token **digit** has a synthesized attribute *lexval* whose value is assumed to be supplied by the lexical analyzer. The rule associated with the production $L \rightarrow E \text{ n}$ for the starting nonterminal *L* is just a procedure that prints as output the value of the arithmetic expression generated by *E*; we can think of this rule as defining a dummy attribute for the nonterminal *L*. A Yacc specification for this desk calculator was presented in Fig. 4.56 to illustrate translation during LR parsing. □

In a syntax-directed definition, terminals are assumed to have synthesized attributes only, as the definition does not provide any semantic rules for terminals. Values for attributes of terminals are usually supplied by the lexical analyzer, as discussed in Section 3.1. Furthermore, the start symbol is assumed not to have any inherited attributes, unless otherwise stated.

Synthesized Attributes

Synthesized attributes are used extensively in practice. A syntax-directed definition that uses synthesized attributes exclusively is said to be an *S-attributed definition*. A parse tree for an S-attributed definition can always be annotated by evaluating the semantic rules for the attributes at each node

bottom up, from the leaves to the root. Section 5.3 describes how an LR-parser generator can be adapted to mechanically implement an S-attributed definition based on an LR grammar.

Example 5.2. The S-attributed definition in Example 5.1 specifies a desk calculator that reads an input line containing an arithmetic expression involving digits, parentheses, the operators $+$ and $*$, followed by a newline character n , and prints the value of the expression. For example, given the expression $3+5+4$ followed by a newline, the program prints the value 19. Figure 5.3 contains an annotated parse tree for the input $3+5+4n$. The output, printed at the root of the tree, is the value of $E.val$ at the first child of the root.

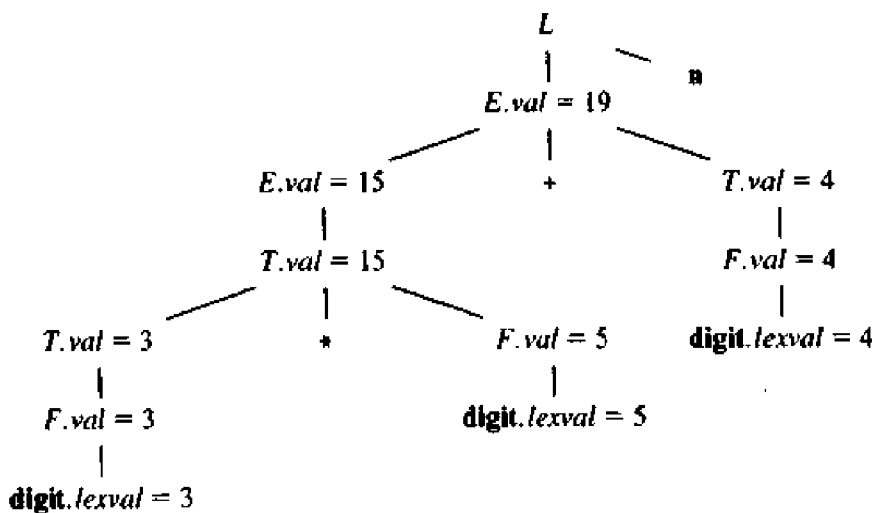


Fig. 5.3. Annotated parse tree for $3+5+4n$.

To see how attribute values are computed, consider the leftmost bottommost interior node, which corresponds to the use of the production $F \rightarrow \text{digit}$. The corresponding semantic rule, $F.val := \text{digit.lexval}$, defines the attribute $F.val$ at that node to have the value 3 because the value of digit.lexval at the child of this node is 3. Similarly, at the parent of this F -node, the attribute $T.val$ has the value 3.

Now consider the node for the production $T \rightarrow T * F$. The value of the attribute $T.val$ at this node is defined by

PRODUCTION	SEMANTIC RULE
$T \rightarrow T_1 * F$	$T.val := T_1.val \times F.val$

When we apply the semantic rule at this node, $T_1.val$ has the value 3 from the left child and $F.val$ the value 5 from the right child. Thus, $T.val$ acquires the value 15 at this node.

The rule associated with the production for the starting nonterminal $L \rightarrow E n$ prints the value of the expression generated by E . $\square$

Inherited Attributes

An inherited attribute is one whose value at a node in a parse tree is defined in terms of attributes at the parent and/or siblings of that node. Inherited attributes are convenient for expressing the dependence of a programming language construct on the context in which it appears. For example, we can use an inherited attribute to keep track of whether an identifier appears on the left or right side of an assignment in order to decide whether the address or the value of the identifier is needed. Although it is always possible to rewrite a syntax-directed definition to use only synthesized attributes, it is often more natural to use syntax-directed definitions with inherited attributes.

In the following example, an inherited attribute distributes type information to the various identifiers in a declaration.

Example 5.3. A declaration generated by the nonterminal D in the syntax-directed definition in Fig. 5.4 consists of the keyword **int** or **real**, followed by a list of identifiers. The nonterminal T has a synthesized attribute *type*, whose value is determined by the keyword in the declaration. The semantic rule $L.in := T.type$, associated with production $D \rightarrow TL$, sets inherited attribute $L.in$ to the type in the declaration. The rules then pass this type down the parse tree using the inherited attribute $L.in$. Rules associated with the productions for L call procedure *addtype* to add the type of each identifier to its entry in the symbol table (pointed to by attribute *entry*).

PRODUCTION	SEMANTIC RULES
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{real}$	$T.type := \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in := L.in$ $\text{addtype}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, L.in)$

Fig. 5.4. Syntax-directed definition with inherited attribute $L.in$.

Figure 5.5 shows an annotated parse tree for the sentence **real** $\text{id}_1, \text{id}_2, \text{id}_3$. The value of $L.in$ at the three L -nodes gives the type of the identifiers id_1, id_2 , and id_3 . These values are determined by computing the value of the attribute $T.type$ at the left child of the root and then evaluating $L.in$ top-down at the three L -nodes in the right subtree of the root. At each L -node we also call the procedure *addtype* to insert into the symbol table the fact that the identifier at the right child of this node has type **real**. $\square$

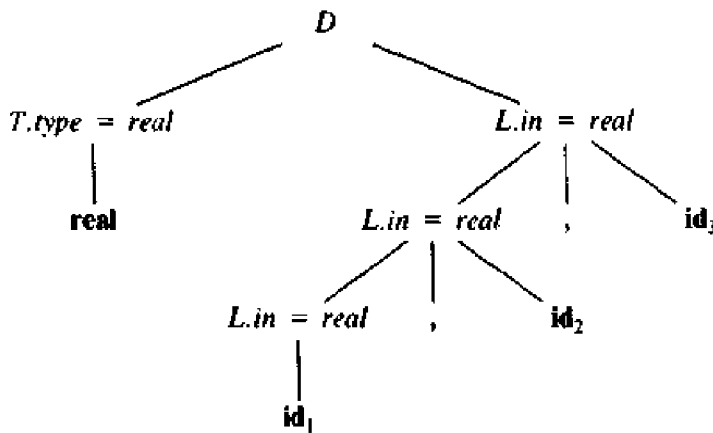


Fig. 5.5. Parse tree with inherited attribute *in* at each node labeled *L*.

Dependency Graphs

If an attribute *b* at a node in a parse tree depends on an attribute *c*, then the semantic rule for *b* at that node must be evaluated after the semantic rule that defines *c*. The interdependencies among the inherited and synthesized attributes at the nodes in a parse tree can be depicted by a directed graph called a *dependency graph*.

Before constructing a dependency graph for a parse tree, we put each semantic rule into the form $b := f(c_1, c_2, \dots, c_k)$, by introducing a dummy synthesized attribute *b* for each semantic rule that consists of a procedure call. The graph has a node for each attribute and an edge to the node for *b* from the node for *c* if attribute *b* depends on attribute *c*. In more detail, the dependency graph for a given parse tree is constructed as follows.

```

for each node n in the parse tree do
    for each attribute a of the grammar symbol at n do
        construct a node in the dependency graph for a;
for each node n in the parse tree do
    for each semantic rule  $b := f(c_1, c_2, \dots, c_k)$ 
        associated with the production used at n do
        for i := 1 to k do
            construct an edge from the node for ci to the node for b;
  
```

For example, suppose $A.a := f(X.x, Y.y)$ is a semantic rule for the production $A \rightarrow XY$. This rule defines a synthesized attribute *A.a* that depends on the attributes *X.x* and *Y.y*. If this production is used in the parse tree, then there will be three nodes *A.a*, *X.x*, and *Y.y* in the dependency graph with an edge to *A.a* from *X.x* since *A.a* depends on *X.x*, and an edge to *A.a* from *Y.y* since *A.a* also depends on *Y.y*.

If the production $A \rightarrow XY$ has the semantic rule $X.i := g(A.a, Y.y)$ associated with it, then there will be an edge to *X.i* from *A.a* and also an edge to *X.i* from *Y.y*, since *X.i* depends on both *A.a* and *Y.y*.

Example 5.4. Whenever the following production is used in a parse tree, we add the edges shown in Fig. 5.6 to the dependency graph.

PRODUCTION	SEMANTIC RULE
$E \rightarrow E_1 + E_2$	$E.val := E_1.val + E_2.val$

The three nodes of the dependency graph marked by $\bullet$ represent the synthesized attributes $E.val$, $E_1.val$, and $E_2.val$ at the corresponding nodes in the parse tree. The edge to $E.val$ from $E_1.val$ shows that $E.val$ depends on $E_1.val$ and the edge to $E.val$ from $E_2.val$ shows that $E.val$ also depends on $E_2.val$. The dotted lines represent the parse tree and are not part of the dependency graph. $\square$

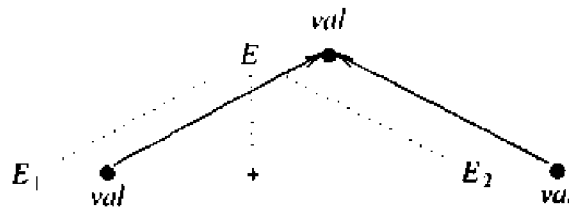


Fig. 5.6. $E.val$ is synthesized from $E_1.val$ and $E_2.val$.

Example 5.5. Figure 5.7 shows the dependency graph for the parse tree in Fig. 5.5. Nodes in the dependency graphs are marked by numbers; these numbers will be used below. There is an edge to node 5 for $L.in$ from node 4 for $T.type$ because the inherited attribute $L.in$ depends on the attribute $T.type$ according to the semantic rule $L.in := T.type$ for the production $D \rightarrow TL$. The two downward edges into nodes 7 and 9 arise because $L_1.in$ depends on $L.in$ according to the semantic rule $L_1.in := L.in$ for the production $L \rightarrow L_1, id$. Each of the semantic rules $addtype(id.entry, L.in)$ associated with the L -productions leads to the creation of a dummy attribute. Nodes 6, 8, and 10 are constructed for these dummy attributes. $\square$

Evaluation Order

A *topological sort* of a directed acyclic graph is any ordering $m_1, m_2, \dots, m_k$ of the nodes of the graph such that edges go from nodes earlier in the ordering to later nodes; that is, if $m_i \rightarrow m_j$ is an edge from m_i to m_j , then m_i appears before m_j in the ordering.

Any topological sort of a dependency graph gives a valid order in which the semantic rules associated with the nodes in a parse tree can be evaluated. That is, in the topological sort, the dependent attributes $c_1, c_2, \dots, c_k$ in a semantic rule $b := f(c_1, c_2, \dots, c_k)$ are available at a node before f is evaluated.

The translation specified by a syntax-directed definition can be made precise

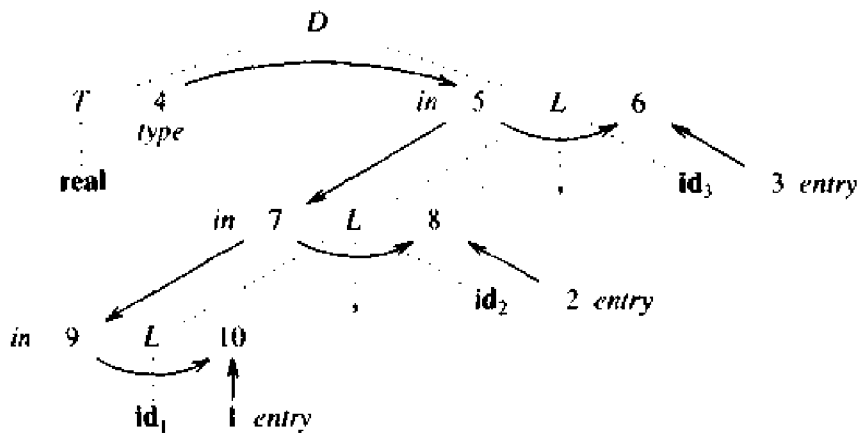


Fig. 5.7. Dependency graph for parse tree of Fig. 5.5.

as follows. The underlying grammar is used to construct a parse tree for the input. The dependency graph is constructed as discussed above. From a topological sort of the dependency graph, we obtain an evaluation order for the semantic rules. Evaluation of the semantic rules in this order yields the translation of the input string.

Example 5.6. Each of the edges in the dependency graph in Fig. 5.7 goes from a lower-numbered node to a higher-numbered node. Hence, a topological sort of the dependency graph is obtained by writing down the nodes in the order of their numbers. From this topological sort, we obtain the following program. We write a_n for the attribute associated with the node numbered n in the dependency graph.

```

 $a_4 := \text{real};$ 
 $a_5 := a_4;$ 
 $\text{addtype}(\text{id}_3.\text{entry}, a_5);$ 
 $a_7 := a_5;$ 
 $\text{addtype}(\text{id}_2.\text{entry}, a_7);$ 
 $a_9 := a_7;$ 
 $\text{addtype}(\text{id}_1.\text{entry}, a_9);$ 

```

Evaluating these semantic rules stores the type *real* in the symbol-table entry for each identifier. □

Several methods have been proposed for evaluating semantic rules:

1. *Parse-tree methods.* At compile time, these methods obtain an evaluation order from a topological sort of the dependency graph constructed from the parse tree for each input. These methods will fail to find an evaluation order only if the dependency graph for the particular parse tree under consideration has a cycle.

2. *Rule-based methods.* At compiler-construction time, the semantic rules associated with productions are analyzed, either by hand, or by a specialized tool. For each production, the order in which the attributes associated with that production are evaluated is predetermined at compiler-construction time.
3. *Oblivious methods.* An evaluation order is chosen without considering the semantic rules. For example, if translation takes place during parsing, then the order of evaluation is forced by the parsing method, independent of the semantic rules. An oblivious evaluation order restricts the class of syntax-directed definitions that can be implemented.

Rule-based and oblivious methods need not explicitly construct the dependency graph at compile time, so they can be more efficient in their use of compile time and space.

A syntax-directed definition is said to be *circular* if the dependency graph for some parse tree generated by its grammar has a cycle. Section 5.10 discusses how to test a syntax-directed definition for circularity.

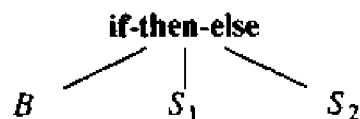
5.2 CONSTRUCTION OF SYNTAX TREES

In this section, we show how syntax-directed definitions can be used to specify the construction of syntax trees and other graphical representations of language constructs.

The use of syntax trees as an intermediate representation allows translation to be decoupled from parsing. Translation routines that are invoked during parsing must live with two kinds of restrictions. First, a grammar that is suitable for parsing may not reflect the natural hierarchical structure of the constructs in the language. For example, a grammar for Fortran may view a subroutine as consisting simply of a list of statements. However, analysis of the subroutine may be easier if we use a tree representation that reflects the nesting of DO loops. Second, the parsing method constrains the order in which nodes in a parse tree are considered. This order may not match the order in which information about a construct becomes available. For this reason, compilers for C usually construct syntax trees for declarations.

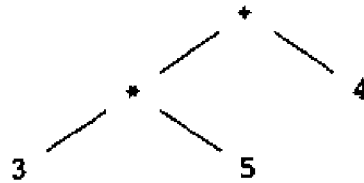
Syntax Trees

An (abstract) syntax tree is a condensed form of parse tree useful for representing language constructs. The production $S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$ might appear in a syntax tree as



In a syntax tree, operators and keywords do not appear as leaves, but rather are associated with the interior node that would be the parent of those leaves

in the parse tree. Another simplification found in syntax trees is that chains of single productions may be collapsed; the parse tree of Fig. 5.3 becomes the syntax tree



Syntax-directed translation can be based on syntax trees as well as parse trees. The approach is the same in each case; we attach attributes to the nodes as in a parse tree.

Constructing Syntax Trees for Expressions

The construction of a syntax tree for an expression is similar to the translation of the expression into postfix form. We construct subtrees for the subexpressions by creating a node for each operator and operand. The children of an operator node are the roots of the nodes representing the subexpressions constituting the operands of that operator.

Each node in a syntax tree can be implemented as a record with several fields. In the node for an operator, one field identifies the operator and the remaining fields contain pointers to the nodes for the operands. The operator is often called the *label* of the node. When used for translation, the nodes in a syntax tree may have additional fields to hold the values (or pointers to values) of attributes attached to the node. In this section, we use the following functions to create the nodes of syntax trees for expressions with binary operators. Each function returns a pointer to a newly created node.

1. *mknode*(*op*, *left*, *right*) creates an operator node with label *op* and two fields containing pointers to *left* and *right*.
2. *mkleaf*(*id*, *entry*) creates an identifier node with label *id* and a field containing *entry*, a pointer to the symbol-table entry for the identifier.
3. *mkleaf*(*num*, *val*) creates a number node with label *num* and a field containing *val*, the value of the number.

Example 5.7. The following sequence of functions calls creates the syntax tree for the expression $a - 4 + c$ in Fig. 5.8. In this sequence, $p_1, p_2, \dots, p_5$ are pointers to nodes, and *entry_a* and *entry_c* are pointers to the symbol-table entries for identifiers *a* and *c*, respectively.

- | | |
|--|--|
| (1) $p_1 := \text{mkleaf}(\text{id}, \text{entry}_a);$ | (4) $p_4 := \text{mkleaf}(\text{id}, \text{entry}_c);$ |
| (2) $p_2 := \text{mkleaf}(\text{num}, 4);$ | (5) $p_5 := \text{mknode}('+', p_3, p_4);$ |
| (3) $p_3 := \text{mknode}('-', p_1, p_2);$ | |

The tree is constructed bottom up. The function calls *mkleaf*(*id*, *entry_a*) and *mkleaf*(*num*, 4) construct the leaves for *a* and 4; the pointers to these

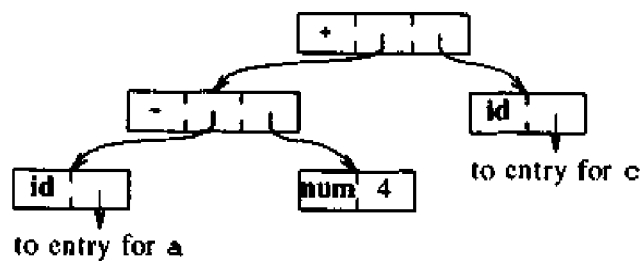


Fig. 5.8. Syntax tree for $a-4+c$.

nodes are saved using p_1 and p_2 . The call $mknnode('-', p_1, p_2)$ then constructs the interior node with the leaves for a and 4 as children. After two more steps, p_3 is left pointing to the root. □

A Syntax-Directed Definition for Constructing Syntax Trees

Figure 5.9 contains an S-attributed definition for constructing a syntax tree for an expression containing the operators $+$ and $-$. It uses the underlying productions of the grammar to schedule the calls of the functions $mknnode$ and $mkleaf$ to construct the tree. The synthesized attribute $nptr$ for E and T keeps track of the pointers returned by the function calls.

PRODUCTION	SEMANTIC RULES
$E \rightarrow E_1 + T$	$E.nptr := mknnode('+', E_1.nptr, T.nptr)$
$E \rightarrow E_1 - T$	$E.nptr := mknnode('-', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr := T.nptr$
$T \rightarrow (E)$	$T.nptr := E.nptr$
$T \rightarrow id$	$T.nptr := mkleaf(id, id.entry)$
$T \rightarrow num$	$T.nptr := mkleaf(num, num.val)$

Fig. 5.9. Syntax-directed definition for constructing a syntax tree for an expression.

Example 5.8. An annotated parse tree depicting the construction of a syntax tree for the expression $a-4+c$ is shown in Fig. 5.10. The parse tree is shown dotted. The parse-tree nodes labeled by the nonterminals E and T use the synthesized attribute $nptr$ to hold a pointer to the syntax-tree node for the expression represented by the nonterminal.

The semantic rules associated with the productions $T \rightarrow id$ and $T \rightarrow num$ define attribute $T.nptr$ to be a pointer to a new leaf for an identifier and a number, respectively. Attributes $id.entry$ and $num.val$ are the lexical values assumed to be returned by the lexical analyzer with the tokens id and num .

In Fig. 5.10, when an expression E is a single term, corresponding to a use

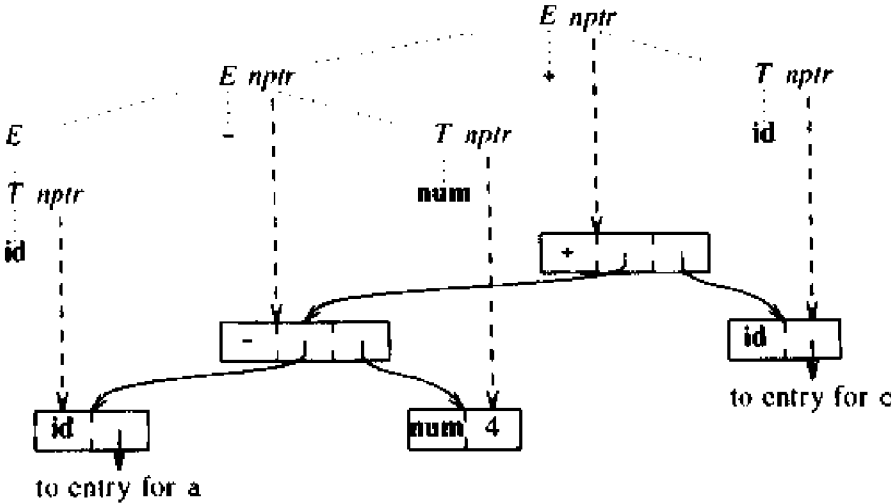


Fig. 5.10. Construction of a syntax-tree for $a-4+c$.

of the production $E \rightarrow T$, the attribute $E.nptr$ gets the value of $T.nptr$. When the semantic rule $E.nptr := mknode('-', E_1.nptr, T.nptr)$ associated with the production $E \rightarrow E_1 - T$ is invoked, previous rules have set $E_1.nptr$ and $T.nptr$ to be pointers to the leaves for a and 4 , respectively.

In interpreting Fig. 5.10, it is important to realize that the lower tree, formed from records is a "real" syntax tree that constitutes the output, while the dotted tree above is the parse tree, which may exist only in a figurative sense. In the next section, we show how an S-attributed definition can be simply implemented using the stack of a bottom-up parser to keep track of attribute values. In fact, with this implementation, the node-building functions are invoked in the same order as in Example 5.7. $\square$

Directed Acyclic Graphs for Expressions

A directed acyclic graph (hereafter called a *dag*) for an expression identifies the common subexpressions in the expression. Like a syntax tree, a dag has a node for every subexpression of the expression; an interior node represents an operator and its children represent its operands. The difference is that a node in a dag representing a common subexpression has more than one "parent;" in a syntax tree, the common subexpression would be represented as a duplicated subtree.

Figure 5.11 contains a dag for the expression

$$a + a * (b - c) + (b - c) * d$$

The leaf for a has two parents because a is common to the two subexpressions a and $a * (b - c)$. Likewise, both occurrences of the common subexpression $b - c$ are represented by the same node, which also has two parents.

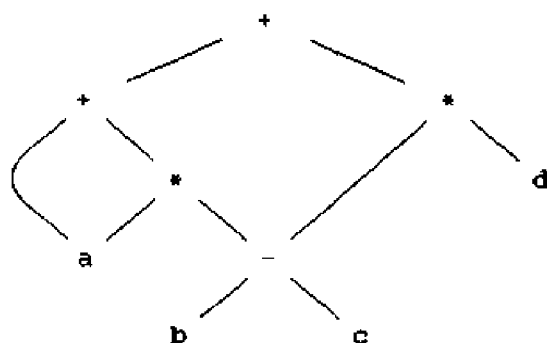


Fig. 5.11. Dag for the expression $a + a * (b - c) + (b - c) * d$.

The syntax-directed definition of Fig. 5.9 will construct a dag instead of a syntax tree if we modify the operations for constructing nodes. A dag is obtained if the function constructing a node first checks to see whether an identical node already exists. For example, before constructing a new node with label *op* and fields with pointers to *left* and *right*, *mknnode*(*op*, *left*, *right*) can check whether such a node has already been constructed. If so, *mknnode*(*op*, *left*, *right*) can return a pointer to the previously constructed node. The leaf-constructing functions *mkleaf* can behave similarly.

Example 5.9. The sequence of instructions in Fig. 5.12 constructs the dag in Fig. 5.11, provided *mknnode* and *mkleaf* create new nodes only when necessary, returning pointers to existing nodes with the correct label and children whenever possible. In Fig. 5.12, *a*, *b*, *c*, and *d* point to the symbol-table entries for identifiers *a*, *b*, *c*, and *d*.

- | | |
|---|---|
| (1) $p_1 := \text{mkleaf}(\text{id}, a);$ | (8) $p_8 := \text{mkleaf}(\text{id}, b);$ |
| (2) $p_2 := \text{mkleaf}(\text{id}, a);$ | (9) $p_9 := \text{mkleaf}(\text{id}, c);$ |
| (3) $p_3 := \text{mkleaf}(\text{id}, b);$ | (10) $p_{10} := \text{mknnode}('-', p_8, p_9);$ |
| (4) $p_4 := \text{mkleaf}(\text{id}, c);$ | (11) $p_{11} := \text{mkleaf}(\text{id}, d);$ |
| (5) $p_5 := \text{mknnode}('-', p_3, p_4);$ | (12) $p_{12} := \text{mknnode}('*', p_{10}, p_{11});$ |
| (6) $p_6 := \text{mknnode}('*', p_2, p_5);$ | (13) $p_{13} := \text{mknnode}('+', p_7, p_{12});$ |
| (7) $p_7 := \text{mknnode}('+', p_1, p_6);$ | |

Fig. 5.12. Instructions for constructing the dag of Fig. 5.11.

When the call *mkleaf*(*id*, *a*) is repeated on line 2, the node constructed by the previous call *mkleaf*(*id*, *a*) is returned, so $p_1 = p_2$. Similarly, the nodes returned on lines 8 and 9 are the same as those returned on lines 3 and 4, respectively. Hence, the node returned on line 10 must be the same one constructed by the call of *mknnode* on line 5. $\square$

In many applications, nodes are implemented as records stored in an array, as in Fig. 5.13. In the figure, each record has a label field that determines the

nature of the node. We can refer to a node by its index or position in the array. The integer index of a node is often called a *value number* for historical reasons. For example, using value numbers, we can say node 3 has label $+$, its left child is node 1, and its right child is node 2. The following algorithm can be used to create nodes for a dag representation of an expression.

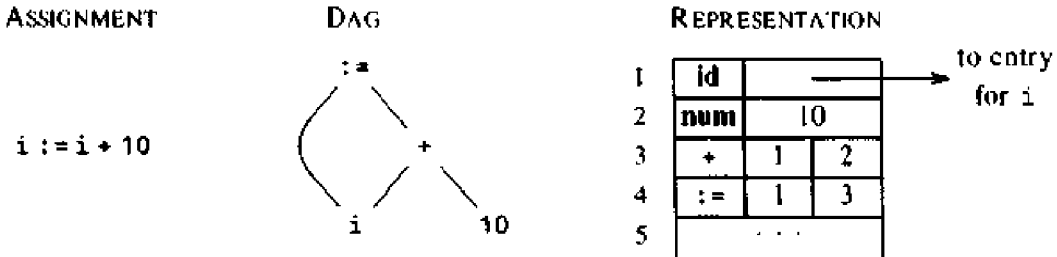


Fig. 5.13. Nodes in a dag for $i := i + 10$ allocated from an array.

Algorithm 5.1. Value-number method for constructing a node in a dag.

Suppose that nodes are stored in an array, as in Fig. 5.13, and that each node is referred to by its value number. Let the *signature* of an operator node be a triple $\langle op, l, r \rangle$ consisting of its label op , left child l , and right child r .

Input. Label op , node l , and node r .

Output. A node with signature $\langle op, l, r \rangle$.

Method. Search the array for a node m with label op , left child l , and right child r . If there is such a node, return m ; otherwise, create a new node n with label op , left child l , right child r , and return n .

An obvious way to determine if node m is already in the array is to keep all previously created nodes on a list and to check each node on the list to see if it has the desired signature. The search for m can be made more efficient by using k lists, called buckets, and using a hashing function h to determine which bucket to search.¹

The hash function h computes the number of a bucket from the value of op , l , and r . It will always return the same bucket number, given the same arguments. If m is not in the bucket $h(op, l, r)$, then a new node n is created and added to this bucket, so subsequent searches will find it there. Several signatures may hash into the same bucket number, but in practice we expect each bucket to contain a small number of nodes.

Each bucket can be implemented as a linked list as shown in Fig. 5.14.

¹ Any data structure that implements dictionaries in the sense of Aho, Hopcroft, and Ullman [1983] suffices. The important property of the structure is that given a key, i.e., a label op and two nodes l and r , we can rapidly obtain a node m with signature $\langle op, l, r \rangle$, or determine that none exists.